R Programming Notes for Absolute Beginners

Part 3: Matrices, Arrays, Lists, Factors, Data Frames and Tibbles

A friendly, picture-filled guide to the containers R uses to hold data. Every code line is explained in plain words, with diagrams, charts, practice questions, a comparison of all the structures, and free places to keep learning.

Contents

(If the contents list looks empty, right-click it in Word and choose Update Field.)

1. Welcome Back: The Map

In the last notes you met the vector: a row of boxes holding values of one type. Now we meet its relatives. Think of them as different containers in a kitchen: a tray with rows and columns (matrix), a stack of trays (array), a toolbox with mixed contents (list), a labelled jar of categories (factor), and a spreadsheet (data frame). Last comes the tibble, a spreadsheet with better manners.

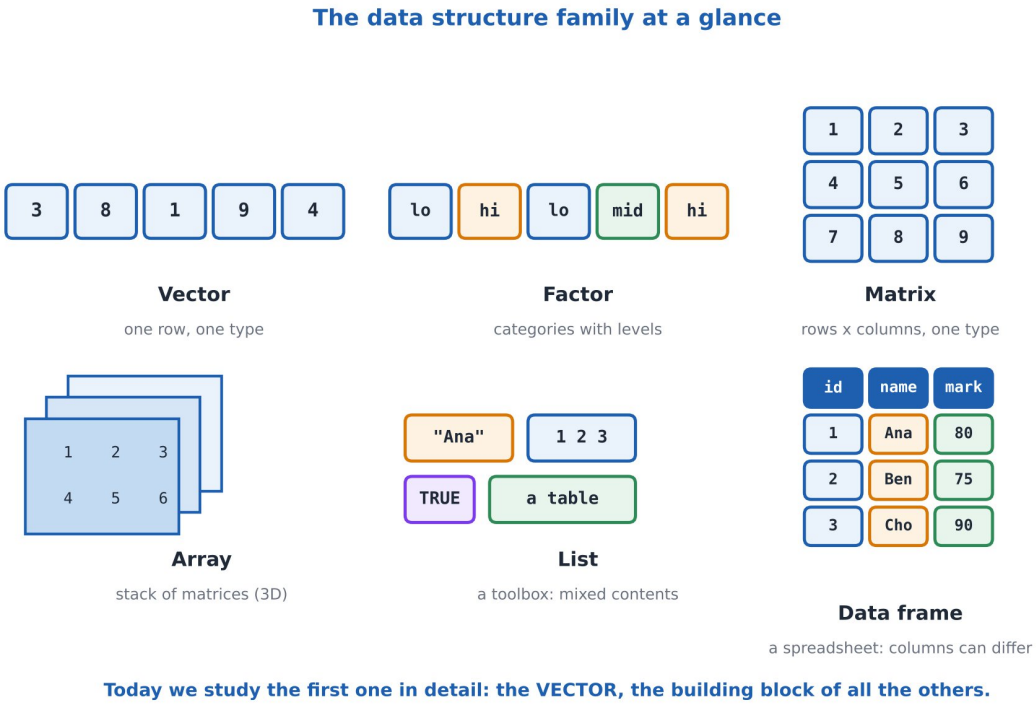


Figure 1: The data structure family. Today we meet all of them.

> **In plain words:** We take them in a gentle order: matrix, array, list, factor, data frame, tibble. Each one builds on the one before. By the end you will also know how to choose between them.

1.1 What you can do with each one

For every structure we follow the same routine, so it becomes familiar:

- **Create it** and look at it.
- **Check it:** what is it, how big is it, what types are inside? (class, dim, length, str)
- **Pick things out of it:** accessing rows, columns, items.
- **Change it:** modify, add, delete.
- **Do maths with it:** arithmetic, summaries, comparisons.
- **Avoid the classic mistakes.**

1.2 A note on the examples

Many online tutorials contain small slips that confuse beginners (for example, examples that claim to change a list but actually only print a copy). In these notes we have corrected them and pointed out the traps, so what you type does what the explanation says.

> **Tip:** As before, run the code in order from top to bottom, and type it yourself. A ready-made R file with all the examples, and a separate practice file with exercises, come with this document.

2. Matrices

2.1 What is a matrix?

A matrix is a two-dimensional grid of values arranged in rows (going across) and columns (going down). Like a vector, every value in a matrix must be the same type. Picture a chessboard, an egg tray, or a seating plan in a cinema.

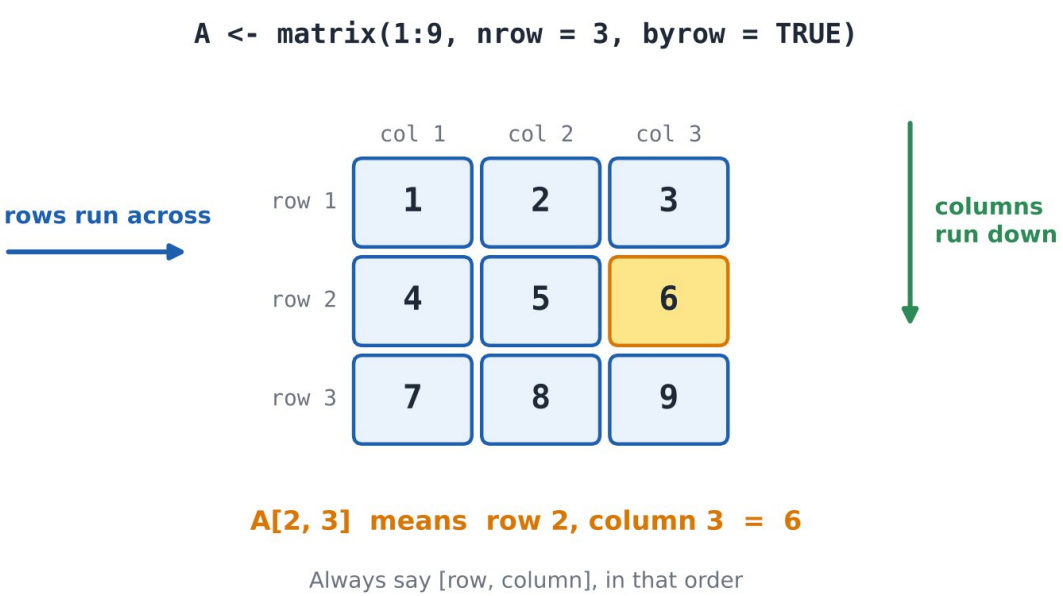


Figure 2: A matrix is a grid. We always write [row, column] in that order.

2.2 Creating a matrix

We use the matrix() function. You give it the values, and how many rows (nrow) or columns (ncol) you want. By default R fills the grid column by column. Add byrow = TRUE to fill it row by row instead.

Code (type this)	What it does, in simple words	What you will see
`A <- matrix(1:9, nrow = 3, byrow = TRUE)`	Makes a grid from the numbers 1 to 9 with 3 rows. byrow = TRUE means "fill across each row first". It is saved in the box named A.	*(nothing is shown)*

`A`	Look at the matrix. R labels rows as [1,] [2,] [3,] and columns as [,1] [,2] [,3].	`[,1] [,2] [,3]`
		`[1,] 1 2 3`
		`[2,] 4 5 6`
		`[3,] 7 8 9`

`B <- matrix(1:6, nrow = 2)`	No byrow, so R fills down the columns (the default).	*(nothing is shown)*

`B`	1 and 2 went in column 1, 3 and 4 in column 2, 5 and 6 in column 3.	`[,1] [,2] [,3]`
		`[1,] 1 3 5`
		`[2,] 2 4 6`

`matrix(1:6, nrow = 2, byrow = TRUE)`	The same numbers, filled across the rows instead.	`[,1] [,2] [,3]`
		`[1,] 1 2 3`
		`[2,] 4 5 6`

`matrix(0, nrow = 2, ncol = 2)`	If you give just one value, R repeats it	`[,1] [,2]`

Code (type this)	What it does, in simple words	What you will see
	to fill the grid. A handy way to start with all zeros.	`[1,] 0 0` `[2,] 0 0`
`M <- matrix(1:9, nrow = 3, byrow = TRUE,` `  dimnames = list(c("r1", "r2", "r3"), c("x", "y", "z")))`	dimnames gives names to the rows (first list item) and the columns (second list item). Names make the matrix easier to read.	*(nothing is shown)*

`M`	The labels replace the [1,] and [,1] numbering.	`  x y z`
		`r1 1 2 3`
		`r2 4 5 6`
		`r3 7 8 9`

`M["r2", "y"]`	Pick an item by its row name and column name.	`[1] 5`
`M["r1", ]`	The whole of row r1. Leaving the column blank means "all columns".	`x y z` `1 2 3`

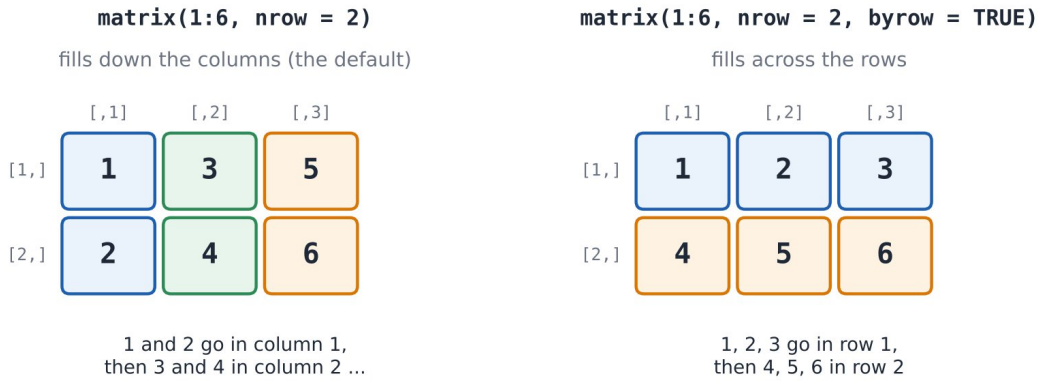


Figure 3: byrow = FALSE (the default) fills down columns; byrow = TRUE fills across rows.

Special matrices

Code (type this)	What it does, in simple words	What you will see

`matrix(5, 3, 3)`	A 3 by 3 matrix where every cell is 5. The three numbers are: the value, the rows, the columns.	`     [,1] [,2] [,3]`
		`[1,]    5    5    5`
		`[2,]    5    5    5`
		`[3,]    5    5    5`

`diag(c(5, 3, 3))`	A diagonal matrix: your numbers go down the main diagonal (top-left to bottom-right) and every other cell is 0.	`     [,1] [,2] [,3]`
		`[1,]    5    0    0`
		`[2,]    0    3    0`
		`[3,]    0    0    3`

`diag(3)`	The identity matrix: ones down the diagonal, zeros elsewhere. It plays the role of the number 1 in matrix maths.	`     [,1] [,2] [,3]`
		`[1,]    1    0    0`
		`[2,]    0    1    0`
		`[3,]    0    0    1`

2.3 Checking your matrix

Before working with data, always ask: what is this, and how big is it? These functions answer those questions.

Code (type this)	What it does, in simple words	What you will see
`class(A)`	What kind of object is A? A matrix is also a special kind of array, so R gives two names.	`[1] "matrix" "array"`
`is.matrix(A)`	A yes/no check.	`[1] TRUE`
`typeof(A)`	What type of values are inside? Whole numbers, so "integer".	`[1] "integer"`
`dim(A)`	dim means dimensions: the number of rows, then columns.	`[1] 3 3`
`nrow(A)`	Number of rows.	`[1] 3`
`ncol(A)`	Number of columns.	`[1] 3`
`length(A)`	How many values altogether (3 x 3 = 9).	`[1] 9`
`str(A)`	A short summary. Notice the numbers are listed column by column: R stores every matrix down its columns underneath, even if you filled it by row.	
`int [1:3, 1:3] 1 4 7 2 5 8 3 6 9`		
`is.numeric(A)`	Are the values numbers?	`[1] TRUE`
`anyNA(A)`	Is there any missing value (NA) in the matrix?	`[1] FALSE`

2.4 Accessing parts of a matrix

Use square brackets with a comma: matrix[row, column]. The number before the comma picks rows, the number after picks columns. Leave one side empty to mean "all of them".

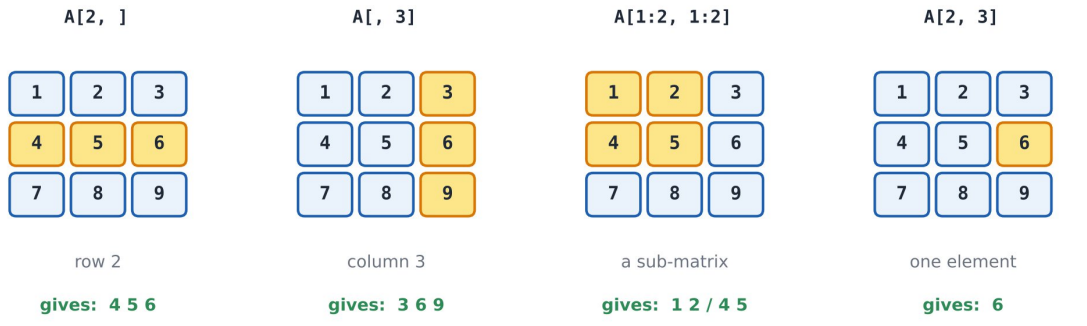


Figure 4: Picking rows, columns, a sub-matrix and a single element.

Code (type this)	What it does, in simple words	What you will see
`A[2, ]`	Row 2, all columns (nothing after the	`[1] 4 5 6`

Code (type this)	What it does, in simple words	What you will see
	comma).	
`A[, 3]`	Column 3, all rows (nothing before the comma).	`[1] 3 6 9`
`A[2, 3]`	One element: row 2, column 3.	`[1] 6`

`A[1:2, 1:2]`	A sub-matrix: rows 1 to 2 and columns 1 to 2. The colon means "from ... to".	`     [,1] [,2]`
		`[1,]    1    2`
		`[2,]    4    5`

`A[-1, ]`	A minus sign means "all except". Everything except row 1.	`     [,1] [,2] [,3]`
		`[1,]    4    5    6`
		`[2,]    7    8    9`

`A[2, , drop = FALSE]`	Normally a single row turns into a plain vector. drop = FALSE keeps it as a matrix with one row.	`     [,1] [,2] [,3]` `[1,]    4    5    6`
`A[A > 5]`	Pick by condition: all values bigger than 5. R reads the matrix down the columns, so the order is 7, 8, 6, 9.	`[1] 7 8 6 9`
`which(A == 5)`	Where is the number 5? The answer is its position counting down the columns: the 5th cell.	`[1] 5`
`A[5]`	A single number inside the brackets counts cells down the columns too.	`[1] 5`

2.5 Changing values in a matrix

To change a value, point to it and assign a new one with <-. We work on a copy called A2 so the original A stays safe for later examples.

Code (type this)	What it does, in simple words	What you will see
`A2 <- A`	Make a copy of A. Changes to A2 will not touch A.	*(nothing is shown)*
`A2[3, 3] <- 30`	Replace the value in row 3, column 3 with 30.	*(nothing is shown)*
`A2[2, ] <- 0`	Set the whole of row 2 to zero. One value is recycled across the row.	*(nothing is shown)*

`A2`	See the result.	`     [,1] [,2] [,3]`
		`[1,]    1    2    3`
		`[2,]    0    0    0`
		`[3,]    7    8   30`

`A2[A2 > 20] <- NA`	Change by condition: every value above 20 becomes NA (missing).	*(nothing is shown)*

`A2`	The 30 is now NA.	`     [,1] [,2] [,3]`
		`[1,]    1    2    3`
		`[2,]    0    0    0`
		`[3,]    7    8   NA`

`A[3, 3]`	The original A is untouched.	`[1] 9`

2.6 Adding rows and columns

To add a row at the bottom use rbind() (r for row, bind means stick together). To add a column on the right use cbind() (c for column). The new piece must fit: a new row needs one value for every column, and a new column needs one value for every row.

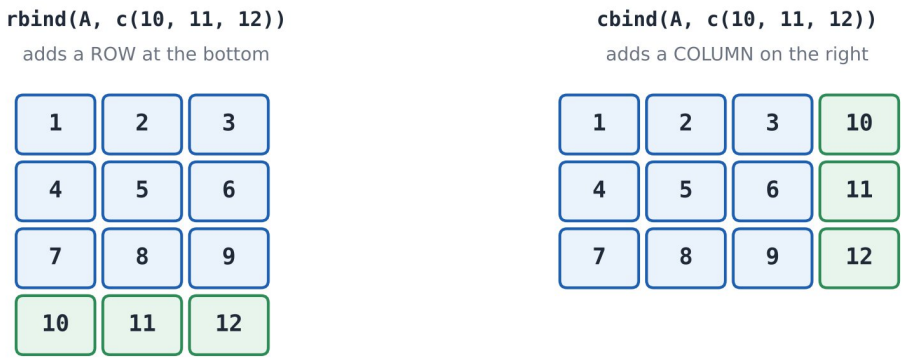


Figure 5: rbind adds a row, cbind adds a column.

Code (type this)	What it does, in simple words	What you will see

`rbind(A, c(10, 11, 12))`	Stick a new row (10, 11, 12) under A. We did not save it, so A itself is unchanged.	`[,1] [,2] [,3]`
		`[1,] 1 2 3`
		`[2,] 4 5 6`
		`[3,] 7 8 9`
		`[4,] 10 11 12`

`cbind(A, c(10, 11, 12))`	Stick a new column (10, 11, 12) on the right.	

`[,1] [,2] [,3] [,4]`
`[1,] 1 2 3 10`
`[2,] 4 5 6 11`
`[3,] 7 8 9 12`		

`rbind(A[1, ], c(10, 11, 12), A[-1, ])`	Insert a row in the middle: first row of A, then the new row, then the rest of A.	`[,1] [,2] [,3]`
		`[1,] 1 2 3`
		`[2,] 10 11 12`
		`[3,] 4 5 6`
		`[4,] 7 8 9`

`A3 <- rbind(A, c(10, 11, 12))`	To keep the change, save the result under a name.	*(nothing is shown)*
`dim(A3)`	Now 4 rows and 3 columns.	`[1] 4 3`
`cbind(A, matrix(c(10, 11, 12), nrow = 1))`	MISTAKE: the new piece is a 1-row matrix, but A has 3 rows. They do not fit side by side. (Left out of the ready-made R file.)	
`Error in cbind(A, matrix(c(10, 11, 12), nrow = 1)) :` `number of rows of matrices must match (see arg 2)`		

2.7 Deleting rows and columns

Use the minus sign to leave something out, then save the result back. Remember: leaving it out only shows a smaller matrix until you assign it.

Code (type this)	What it does, in simple words	What you will see

`A[-2, ]`	Everything except row 2.	`[,1] [,2] [,3]`
		`[1,] 1 2 3`
		`[2,] 7 8 9`

`A[, -2]`	Everything except column 2.	`[,1] [,2]`
		`[1,] 1 3`
		`[2,] 4 6`
		`[3,] 7 9`

`A4 <- A[-2, ]`	Save the smaller matrix as A4. Now the row is "deleted" from A4.	*(nothing is shown)*
`dim(A4)`	Two rows left.	`[1] 2 3`

2.8 Arithmetic with matrices

Matrices follow the same idea as vectors: maths happens on every cell at once. There are two kinds of multiplication, and mixing them up is a very common mistake. The star * multiplies cell by cell. The symbol %*% does true matrix multiplication (rows times columns).

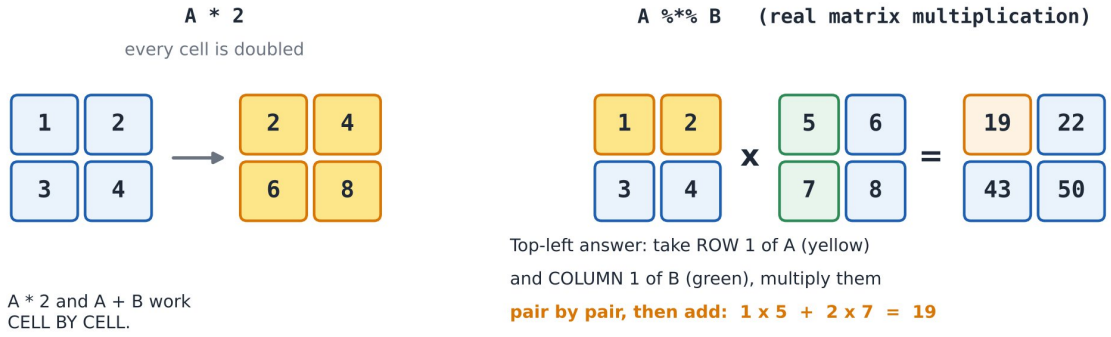


Figure 6: Cell-by-cell maths versus real matrix multiplication.

Code (type this)	What it does, in simple words	What you will see

`A + 10`	Add 10 to every cell.	`[,1] [,2] [,3]`
		`[1,] 11 12 13`
		`[2,] 14 15 16`
		`[3,] 17 18 19`

`A * 2`	Double every cell.	`[,1] [,2] [,3]`
		`[1,] 2 4 6`
		`[2,] 8 10 12`
		`[3,] 14 16 18`

`B2 <- matrix(9:1, nrow = 3, byrow = TRUE)`	A second 3 by 3 matrix, counting down from 9 to 1.	*(nothing is shown)*

`A + B2`	Add two matrices of the same size, cell by cell. Every cell comes out as 10, because 1+9, 2+8, 3+7 ...	`[,1] [,2] [,3]`
		`[1,] 10 10 10`
		`[2,] 10 10 10`
		`[3,] 10 10 10`

`A * B2`	Multiply cell by cell (NOT matrix	`[,1] [,2] [,3]`

Code (type this)	What it does, in simple words	What you will see

	multiplication).	`[1,]  9  16  21`
		`[2,]  24  25  24`
		`[3,]  21  16  9`

`A %*% B2`	True matrix multiplication: each answer is a row of A multiplied pair by pair with a column of B2, then added. The top-left value is 1x9 + 2x6 + 3x3 = 30.	`[,1] [,2] [,3]`
		`[1,]  30  24  18`
		`[2,]  84  69  54`
		`[3,]  138  114  90`

`t(A)`	Transpose: swap rows and columns. Row 1 becomes column 1, and so on.	`[,1] [,2] [,3]`
		`[1,]  1  4  7`
		`[2,]  2  5  8`
		`[3,]  3  6  9`

`P <- matrix(1:4, nrow = 2, byrow = TRUE)`	A small 2 by 2 matrix for the next lines: rows (1, 2) and (3, 4).	*(nothing is shown)*
`Q <- matrix(5:8, nrow = 2, byrow = TRUE)`	Another one: rows (5, 6) and (7, 8).	*(nothing is shown)*

`P %*% Q`	The example from the picture. Top-left: 1x5 + 2x7 = 19.	`[,1] [,2]`
		`[1,]  19  22`
		`[2,]  43  50`

`det(P)`	The determinant of P: 1x4 minus 2x3. A number that tells you if the matrix can be inverted.	`[1] -2`

`solve(P)`	The inverse matrix of P. (Multiplying a matrix by its inverse gives the identity matrix.)	`[,1] [,2]`
		`[1,] -2.0  1.0`
		`[2,]  1.5 -0.5`

`A > 5`	Compare every cell with 5. The answer is a matrix of TRUE/FALSE.	`[,1]  [,2]  [,3]`
		`[1,] FALSE FALSE FALSE`
		`[2,] FALSE FALSE  TRUE`
		`[3,]  TRUE  TRUE  TRUE`

`sum(A)`	Add up every value.	`[1] 45`
`mean(A)`	The average of all nine values.	`[1] 5`
`max(A)`	The biggest value.	`[1] 9`

2.9 Summaries by row and by column

Code (type this)	What it does, in simple words	What you will see
`rowSums(A)`	Add up each row.	`[1]  6 15 24`
`colSums(A)`	Add up each column.	`[1] 12 15 18`
`rowMeans(A)`	Average of each row.	`[1] 2 5 8`
`colMeans(A)`	Average of each column.	`[1] 4 5 6`
`apply(A, 1, max)`	apply() runs a function on each row (the 1) or each column (the 2). Here: the biggest value in each row.	`[1] 3 6 9`
`apply(A, 2, min)`	The smallest value in each column (2 means columns).	`[1] 1 2 3`
`diag(A)`	Pull out the main diagonal.	`[1] 1 5 9`

Code (type this)	What it does, in simple words	What you will see
`sum(diag(A))`	Add up the diagonal (called the trace of the matrix).	`[1] 15`

2.10 Matrices hold one type only

Code (type this)	What it does, in simple words	What you will see

`matrix(c(1, "a", 3, 4), nrow = 2)`	One piece of text in the mix, so R turns every value into text. Just like with vectors, text wins.	`     [,1] [,2]`
		`[1,] "1"  "3"`
		`[2,] "a"  "4"`

2.11 Common matrix mistakes

Code (type this)	What it does, in simple words	What you will see

`matrix(1:5, nrow = 2)`	MISTAKE: 5 values do not fill a 2-row grid evenly. R recycles the first value to fill the gap and warns you (data length is not a multiple of the number of rows).	`     [,1] [,2] [,3]`
		`[1,]    1    3    5`
		`[2,]    2    4    1`

`A[3, 1]`	Remember the order is [row, column]. This is row 3, column 1.	`[1] 7`
`A[1, 3]`	This is row 1, column 3: a different cell! Swapping the two numbers gives a different answer.	`[1] 3`

`A * A`	MISTAKE: expecting this to be matrix multiplication. It squares each cell. For the real thing use A %*% A.	`     [,1] [,2] [,3]`
		`[1,]    1    4    9`
		`[2,]   16   25   36`
		`[3,]   49   64   81`

3. Arrays

3.1 What is an array?

If a matrix is one tray, an array is a stack of trays. It is a container with three or more dimensions: rows, columns, and layers (sometimes called pages or slices). All the values are again the same type.

Here is the family tree: a vector is a one-dimensional array, a matrix is a two-dimensional array, and an array can go to three dimensions or more. Adding, removing and changing items in a one-dimensional array works exactly like the vector operations in the previous notes.

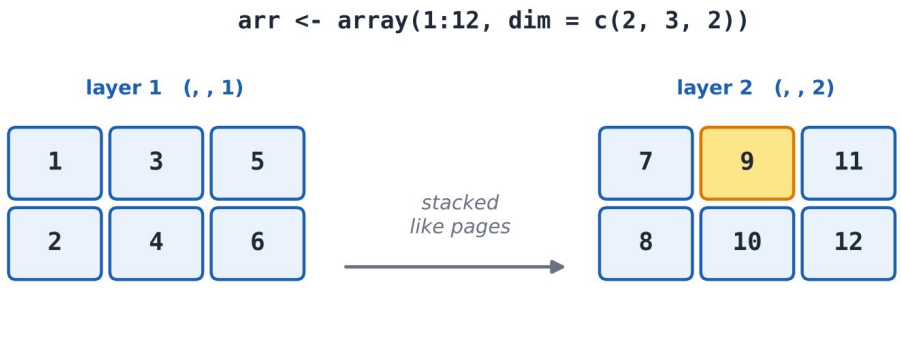


Figure 7: A 2 x 3 x 2 array is two layers of a 2 by 3 grid.

Code (type this)	What it does, in simple words	What you will see
`arr <- array(1:12, dim = c(2, 3, 2))`	array() takes the values and the dimensions. dim = c(2, 3, 2) means 2 rows, 3 columns and 2 layers: 2 x 3 x 2 = 12 values.	*(nothing is shown)*

`arr`	R prints one grid per layer. The label ", , 1" means "layer 1".	`, , 1`
		
		`     [,1] [,2] [,3]`
		`[1,]    1    3    5`
		`[2,]    2    4    6`
		
		`, , 2`
		
		`     [,1] [,2] [,3]`
		`[1,]    7    9   11`
		`[2,]    8   10   12`

`arr_n <- array(1:12, dim = c(2, 3, 2),` `  dimnames = list(c("row1", "row2"), c("col1", "col2", "col3"), c("Arr1", "Arr2")))`	The same array, but with names for the rows, the columns and the layers (three name lists, in that order).	*(nothing is shown)*

`arr_n`	Names replace the numbers.	`, , Arr1`
		
		`     col1 col2 col3`

Code (type this)	What it does, in simple words	What you will see
		row1 1 3 5 row2 2 4 6 , , Arr2 col1 col2 col3 row1 7 9 11 row2 8 10 12

3.2 Checking an array

Code (type this)	What it does, in simple words	What you will see
`dim(arr)`	Rows, columns, layers.	`[1] 2 3 2`
`length(arr)`	Total number of values: 2 x 3 x 2.	`[1] 12`
`class(arr)`	The class is array.	`[1] "array"`
`is.array(arr)`	A yes/no check.	`[1] TRUE`
`typeof(arr)`	The values are integers.	`[1] "integer"`

3.3 Accessing an array

Use three numbers in the brackets: [row, column, layer]. Leave a place empty to mean "all".

Code (type this)	What it does, in simple words	What you will see
`arr[1, 2, 2]`	Row 1, column 2, layer 2.	`[1] 9`
`arr[, , 1]`	The whole of layer 1, as an ordinary matrix.	[,1] [,2] [,3] [1,] 1 3 5 [2,] 2 4 6
`arr[2, , ]`	Row 2 from every layer. Each layer becomes a column in the answer.	[,1] [,2] [1,] 2 8 [2,] 4 10 [3,] 6 12
`arr[, 2, ]`	Column 2 from every layer.	[,1] [,2] [1,] 3 9 [2,] 4 10
`arr_n["row2", "col3", "Arr1"]`	Pick one value by names instead of numbers.	`[1] 6`
`arr_n[, , "Arr2"]`	Layer Arr2 only.	col1 col2 col3 row1 7 9 11 row2 8 10 12
`arr[arr > 10]`	All values bigger than 10.	`[1] 11 12`

3.4 Changing an array

Code (type this)	What it does, in simple words	What you will see
`arr2 <- arr`	Work on a copy.	*(nothing is shown)*
`arr2[1, 1, 1] <- 100`	Change the single value in row 1, column 1, layer 1.	*(nothing is shown)*
`arr2[1, 1, 1]`	Check it.	`[1] 100`
`arr2[, , 2] <- 0`	Set the whole of layer 2 to zero.	*(nothing is shown)*
`sum(arr2)`	Only layer 1 is left: 100 + 2 + 3 + 4 + 5 + 6.	`[1] 120`

3.5 Arithmetic and summaries with arrays

Code (type this)	What it does, in simple words	What you will see
`sum(arr)`	Add up all 12 values.	`[1] 78`
`mean(arr)`	The average of all values.	`[1] 6.5`
`max(arr)`	The biggest value.	`[1] 12`
`apply(arr, 3, sum)`	apply() with 3 means "do it for each layer". Layer 1 adds to 21, layer 2 to 57.	`[1] 21 57`
`apply(arr, 1, sum)`	With 1: do it for each row (across all layers).	`[1] 36 42`

`apply(arr, c(1, 3), sum)`	With c(1, 3): one answer for each row-and-layer pair.	`     [,1] [,2]`
		`[1,]    9   27`
		`[2,]   12   30`

`arr * 2`	Doubling works on every value (the answer is another array, so we do not print all of it here).	*(nothing is shown)*
`as.vector(arr)`	Flatten the array into a plain vector, reading down the columns, layer by layer.	
`[1]  1  2  3  4  5  6  7  8  9 10 11 12`		

3.6 A common array mistake

Code (type this)	What it does, in simple words	What you will see
`arr[1, 2]`	MISTAKE: only two numbers for a three-dimensional array. R needs one number (or a blank) for each dimension. (Left out of the ready-made R file.)	
`Error in arr[1, 2] : incorrect number of dimensions`		
`arr[1, 2, 1]`	FIX: give all three: row 1, column 2, layer 1.	`[1] 3`

4. Lists

4.1 What is a list?

A vector, matrix and array can each hold only one type. A list is different: it is a toolbox with separate compartments, and each compartment can hold anything: a number, some text, a whole vector, a matrix, even another list or a function. Lists are ordered (the compartments have positions) and one-dimensional.

Lists are everywhere in R. When you fit a statistical model, the result is a list. When you read a data frame, it is secretly a list of columns.

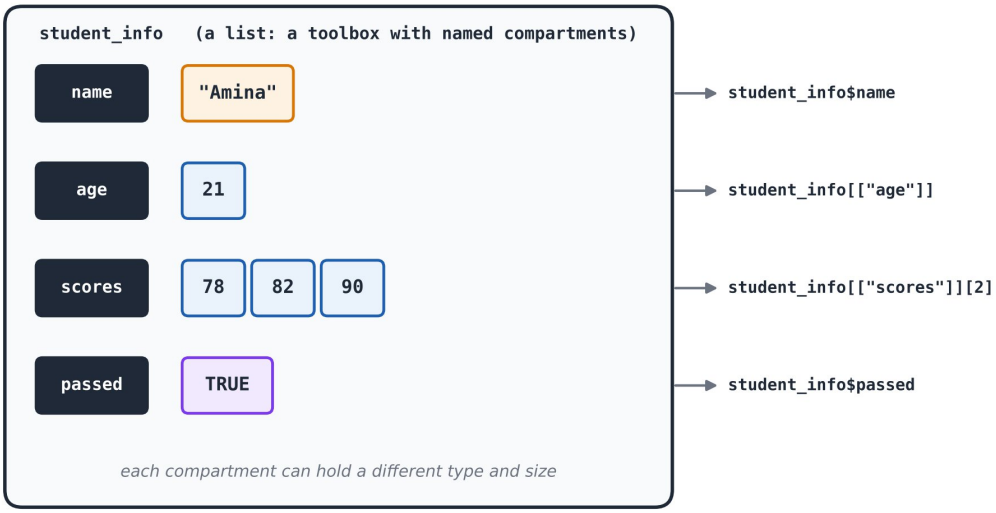


Figure 8: A list is a toolbox. Each compartment has a name and can hold a different type and size.

4.2 Creating a list

Code (type this)	What it does, in simple words	What you will see

`list(1, "a", TRUE)`	The simplest list: a number, a piece of text and a logical, with no names. R numbers the compartments [[1]], [[2]], [[3]].	`[[1]]`
		`[1] 1`
		
		`[[2]]`
		`[1] "a"`
		
		`[[3]]`
		`[1] TRUE`

`student_info <- list(name = "Amina", age = 21,` `  scores = c(78, 82, 90), passed = TRUE)`	A named list. Each item has a name, an equals sign, and its value. Notice scores holds a whole vector in one compartment.	*(nothing is shown)*

`student_info`	R shows each name with a $ sign in front of it.	`$name`
		`[1] "Amina"`
		
		`$age`
		`[1] 21`
		
		`$scores`

Code (type this)	What it does, in simple words	What you will see

		`[1] 78 82 90`
		
		`$passed`
		`[1] TRUE`

> **Tip:** Naming the compartments makes lists much easier to use: student_info$age is far clearer than student_info[[2]].

4.3 Checking a list

Code (type this)	What it does, in simple words	What you will see
`class(student_info)`	What is it? A list.	`[1] "list"`
`is.list(student_info)`	Yes/no check.	`[1] TRUE`
`length(student_info)`	How many compartments? Four (the vector in scores counts as just one).	`[1] 4`
`names(student_info)`	The names of the compartments.	
`[1] "name"  "age"  "scores" "passed"`		
`str(student_info)`	The best way to see what is inside a list at a glance: the type and a preview of each compartment.	

`List of 4`
` $ name  : chr "Amina"`
` $ age  : num 21`
` $ scores: num [1:3] 78 82 90`
` $ passed: logi TRUE`		

`sapply(student_info, class)`	Ask every compartment what type it holds. sapply() applies a function to each item.	
`name  age  scores  passed` `"character"  "numeric"  "numeric"  "logical"`		

4.4 Accessing items: $, [[ ]] and [ ]

There are three ways to reach into a list. Two of them give you the item itself, and one gives you a smaller list. This is the most confusing part of lists, so study the picture.

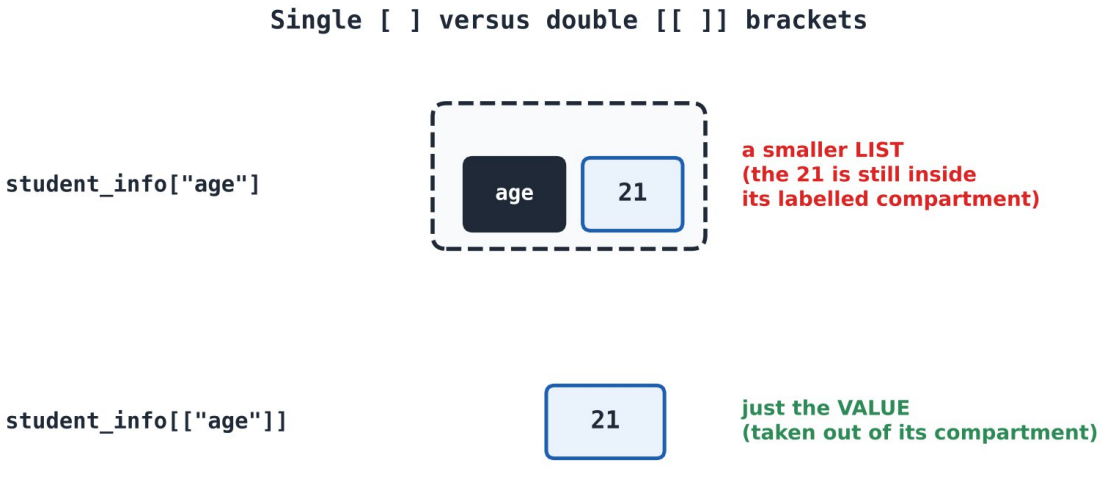


Figure 9: Single brackets return a smaller list; double brackets return what is inside.

Code (type this)	What it does, in simple words	What you will see
`student_info$name`	The dollar sign plus the name gives you the item itself.	`[1] "Amina"`
`student_info[["age"]]`	Double brackets with the name in quotes also give the item itself.	`[1] 21`
`student_info[[3]]`	Double brackets with a position: the 3rd compartment (the scores).	`[1] 78 82 90`
`student_info[[3]][2]`	Go one level deeper: take the 3rd compartment, then its 2nd item.	`[1] 82`
`student_info$scores[2]`	The same thing, written with the dollar sign.	`[1] 82`
`student_info["age"]`	Single brackets return a smaller LIST that still has the compartment name.	`$age` `[1] 21`
`class(student_info["age"])`	Proof: it is still a list.	`[1] "list"`
`class(student_info[["age"]])`	With double brackets you get the plain number.	`[1] "numeric"`

`student_info[c("name", "age")]`	Single brackets can pick several compartments at once, which double brackets cannot.	`$name`
		`[1] "Amina"`
		
		`$age`
		`[1] 21`

In plain words: Easy memory trick: single bracket [ ] gives you the drawer. Double bracket [[ ]] opens the drawer and gives you what is inside.

4.5 Changing, adding and deleting items

Code (type this)	What it does, in simple words	What you will see
`student_info$age <- 22`	Replace an existing item with a new value.	*(nothing is shown)*
`student_info$city <- "Nairobi"`	Use a new name to add a new compartment at the end.	*(nothing is shown)*
`student_info$passed <- NULL`	Assigning NULL to an item deletes it from the list.	*(nothing is shown)*
`student_info[["scores"]][4] <- 95`	Reach inside the scores vector and add a 4th score.	*(nothing is shown)*
`length(student_info)`	Four compartments again: name, age, scores, city.	`[1] 4`
`names(student_info)`	Check the names.	
`[1] "name" "age" "scores" "city"`		
`mean(student_info$scores)`	Use the vector inside the list like any vector: (78 + 82 + 90 + 95) / 4.	`[1] 86.25`
`names(student_info[-1])`	A minus sign works on lists too. This SHOWS the names without the first item, but does not delete it. To really delete you must assign (or use NULL).	
`[1] "age" "scores" "city"`		

4.6 Combining lists

To join two lists use c(). Be careful: c() on two lists gives a longer list, but c() on two vectors gives a vector. Some tutorials mix these up.

Code (type this)	What it does, in simple words	What you will see
`extra <- list(grade = "A")`	A second list with one item.	*(nothing is shown)*
`combined <- c(student_info, extra)`	Join the two lists. The compartments of extra are added to the end.	*(nothing is shown)*
`names(combined)`	Now five compartments.	
`[1] "name" "age" "scores" "city" "grade"`		
`lst1 <- list(1, 2, 3)`	A list of three numbers.	*(nothing is shown)*
`lst2 <- list("Sun", "Mon")`	A list of two pieces of text.	*(nothing is shown)*
`new_list <- c(lst1, lst2)`	Merge into one list of five items.	*(nothing is shown)*

`str(new_list)`	See the five items.	`List of 5`
		` $ : num 1`
		` $ : num 2`
		` $ : num 3`
		` $ : chr "Sun"`
		` $ : chr "Mon"`

`c(c("Debi", "Sandeep"), c(34, 23))`	MISTAKE to watch for: these are vectors, not lists. The result is one flat text vector, and the numbers became text.	
`[1] "Debi" "Sandeep" "34" "23"`		

Code (type this)	What it does, in simple words	What you will see
`nums <- c(1, 5, 6, 3)`	A vector for the next lines.	*(nothing is shown)*
`append(nums, 45)`	append() adds a value at the end. But it only SHOWS the longer result.	`[1]  1  5  6  3 45`
`nums`	nums itself did not change! Save the result to keep it: nums <- append(nums, 45).	`[1] 1 5 6 3`

4.7 Turning a list into other things

Code (type this)	What it does, in simple words	What you will see
`unlist(list(a = 1, b = 2, c = 3))`	unlist() flattens a list into a vector, keeping the names.	`a b c` `1 2 3`
`unlist(list(1:3, 4:5))`	Two vectors inside a list become one long vector.	`[1] 1 2 3 4 5`
`lst <- list(c(1, 2, 3), c(4, 5, 6))`	A list of two vectors.	*(nothing is shown)*

`matrix(unlist(lst), nrow = 2, byrow = TRUE)`	Flatten the list into one vector, then arrange it as a matrix with 2 rows, filled by row.	`     [,1] [,2] [,3]`
		`[1,]    1    2    3`
		`[2,]    4    5    6`

4.8 Doing something to every item: lapply and sapply

Imagine a list of exam scores for three students and you want each student's average. Instead of writing a loop, use sapply(). It applies a function to every compartment and tidies the answers into a vector.

Code (type this)	What it does, in simple words	What you will see
`scores_list <- list(Amina = c(78, 82), Brian = c(61, 65), Chebet = c(88, 90))`	A list with three students, each holding two scores.	*(nothing is shown)*
`sapply(scores_list, mean)`	The average for each student.	`Amina  Brian Chebet` `   80     63     89`
`sapply(scores_list, max)`	The best score for each student.	`Amina  Brian Chebet` `   82     65     90`
`sapply(scores_list, length)`	How many scores each one has.	`Amina  Brian Chebet` `    2      2      2`

4.9 Lists inside lists

Code (type this)	What it does, in simple words	What you will see
`school <- list(name = "Hill School",` `  teachers = list(maths = "Ms Ode", science = "Mr Kim"))`	A list that has another list inside it (the teachers).	*(nothing is shown)*
`school$teachers$maths`	Go in step by step: the school, then its teachers, then the maths teacher.	`[1] "Ms Ode"`

Code (type this)	What it does, in simple words	What you will see
`school[["teachers"]][["science"]]`	The same idea using double brackets.	`[1] "Mr Kim"`

4.10 A secret: data frames are lists

Code (type this)	What it does, in simple words	What you will see
`df_small <- data.frame(a = 1:2, b = c("x", "y"))`	A tiny data frame with two columns.	*(nothing is shown)*
`is.list(df_small)`	A data frame is actually a list in disguise: a list of equal-length columns.	`[1] TRUE`
`typeof(df_small)`	R stores it as a list.	`[1] "list"`
`length(df_small)`	Length counts the columns (list items), not rows.	`[1] 2`

4.11 Common list mistakes

- **Mixing up [ ] and [[ ]].** Remember: [ ] gives a list, [[ ]] gives the thing inside.
- **Forgetting to save.** append(), c() and student_info[-3] show a result but do not change the list unless you assign it back with <-.
- **Deleting with a minus sign and expecting it to stick.** Use student_info$item <- NULL to delete in place.
- **Mixing up c() on lists versus vectors.** c(list1, list2) is a list; c(vector1, vector2) is a vector.
- **Typing the wrong name.** student_info$Age (capital A) returns NULL, not an error, which can hide a bug.

5. Factors

5.1 What is a factor?

A factor is a special vector for categories: values that come from a small, fixed set of groups, like "female / male", "small / medium / large", or "Yes / No / Maybe". R remembers the full list of allowed groups, called levels, and stores each value as a whole-number code pointing to its level.

```
gender <- factor(c("female", "male", "male", "female"))
```

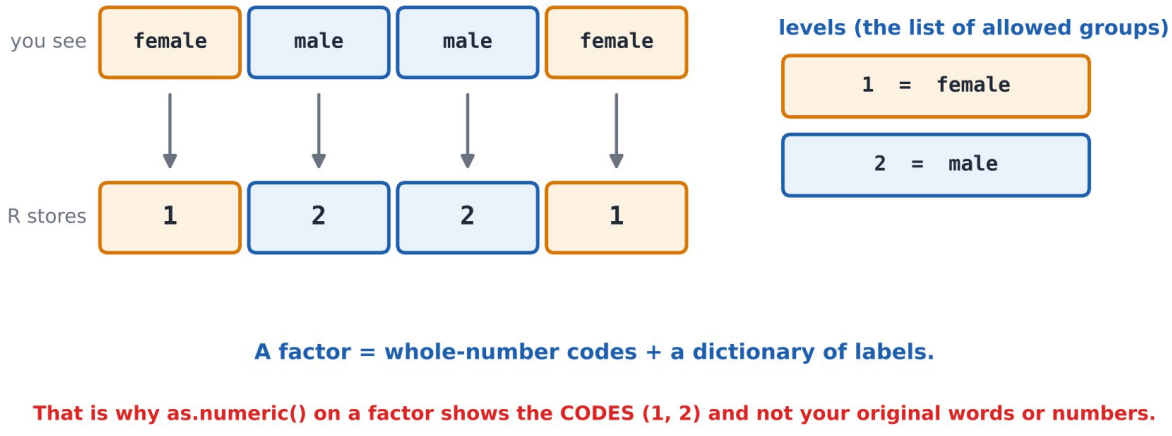


Figure 10: A factor is whole-number codes plus a dictionary of labels (the levels).

5.2 Creating and checking a factor

Code (type this)	What it does, in simple words	What you will see
`gender <- factor(c("female", "male", "male", "female"))`	factor() turns the text into a factor. R finds the different groups itself.	*(nothing is shown)*
`gender`	The values print without quotes, and R lists the levels underneath (alphabetical by default).	
`[1] female male male female` `Levels: female male`		
`levels(gender)`	Just the list of groups.	`[1] "female" "male"`
`nlevels(gender)`	How many groups.	`[1] 2`
`class(gender)`	The type is factor.	`[1] "factor"`
`is.factor(gender)`	A yes/no check.	`[1] TRUE`
`typeof(gender)`	Underneath, a factor is stored as integers.	`[1] "integer"`
`as.integer(gender)`	The hidden codes: female is 1 and male is 2.	`[1] 1 2 2 1`
`as.character(gender)`	Turn it back into ordinary text.	
`[1] "female" "male" "male" "female"`		
`table(gender)`	Count how many items are in each	`gender`

Code (type this)	What it does, in simple words	What you will see
	group. One of the most useful things to do with a factor.	`female male` `2 2`
`summary(gender)`	A quick count per group.	`female male` `2 2`

5.3 Accessing and changing a factor

Code (type this)	What it does, in simple words	What you will see
`gender[3]`	The 3rd item. Notice R still shows the levels.	`[1] male` `Levels: female male`
`gender[c(2, 4)]`	Several items at once.	`[1] male female` `Levels: female male`
`gender[-3]`	Everything except the 3rd item.	
`[1] female male female` `Levels: female male`		
`gender[2] <- "female"`	Change the 2nd value to "female". This works because "female" is an allowed level.	*(nothing is shown)*
`gender`	The 2nd item changed.	
`[1] female female male female` `Levels: female male`		
`g2 <- factor(c("female", "male"))`	A fresh small factor.	*(nothing is shown)*
`g2[1] <- "other"`	MISTAKE: "other" is not one of the allowed levels. R puts NA there and warns you (invalid factor level, NA generated).	*(nothing is shown)*
`g2`	The first item is now missing.	`[1] <NA> male` `Levels: female male`
`levels(g2) <- c(levels(g2), "other")`	FIX: first add the new level to the dictionary.	*(nothing is shown)*
`g2[1] <- "other"`	Now the assignment is allowed.	*(nothing is shown)*
`g2`	The new level is accepted.	
`[1] other male` `Levels: female male other`		

5.4 Ordered factors and tidy-up tools

Sometimes the groups have a natural order: low, medium, high. An ordered factor remembers the order so that you can compare and sort them.

Code (type this)	What it does, in simple words	What you will see
`size <- factor(c("small", "large", "medium", "small"),` `  levels = c("small", "medium", "large"), ordered = TRUE)`	We list the levels in the order we want, and ordered = TRUE tells R that the order matters.	*(nothing is shown)*

Code (type this)	What it does, in simple words	What you will see
`size`	The < signs show the order of the levels.	
`[1] small  large  medium small` `Levels: small < medium < large`		
`size[2] > size[1]`	Comparisons work on ordered factors. Is "large" above "small"?	`[1] TRUE`
`max(size)`	The highest level present.	
`[1] large` `Levels: small < medium < large`		
`sort(size)`	Sorting follows the level order, not the alphabet.	
`[1] small  small  medium large` `Levels: small < medium < large`		
`g3 <- factor(c("a", "b", "c"))[1:2]`	Take only the first two items of a three-level factor.	*(nothing is shown)*
`levels(g3)`	The unused level "c" is still in the dictionary.	`[1] "a" "b" "c"`
`levels(droplevels(g3))`	droplevels() removes levels that are no longer used.	`[1] "a" "b"`
`relevel(gender, ref = "male")`	Choose which level comes first (useful in statistical models).	
`[1] female female male  female` `Levels: male female`		
`cut(c(15, 25, 45), breaks = c(0, 18, 35, 100),` `  labels = c("child", "young", "adult"))`	cut() turns numbers into categories by sorting them into bands: 0-18, 18-35, 35-100.	
`[1] child young adult` `Levels: child young adult`		

5.5 Factors in a data frame, and a famous trap

Code (type this)	What it does, in simple words	What you will see
`df_f <- data.frame(age = c(40, 49, 48),` `  group = factor(c("male", "male", "female")))`	A data frame where the group column is a factor.	*(nothing is shown)*
`is.factor(df_f$group)`	Is the group column a factor?	`[1] TRUE`
`levels(df_f$group)`	Its levels.	`[1] "female" "male"`
`table(df_f$group)`	Count the people in each group.	`female  male` `1  2`
`f <- factor(c("10", "20", "10"))`	A factor that looks like numbers.	*(nothing is shown)*
`as.numeric(f)`	TRAP: this returns the hidden codes (1, 2, 1), not 10, 20, 10.	`[1] 1 2 1`
`as.numeric(as.character(f))`	FIX: turn it into text first, then into numbers.	`[1] 10 20 10`

6. Data Frames

6.1 What is a data frame?

A data frame is R's version of a spreadsheet or a table in a database. It has rows and columns, like a matrix, but with a big difference: every column can hold a different type. One column can be names (text), another marks (numbers), another yes/no (logical).

Here is the key picture to hold in your head: a data frame is a set of vectors of the same length standing side by side. Each column is a vector. Each row is one record (one student, one product, one patient).

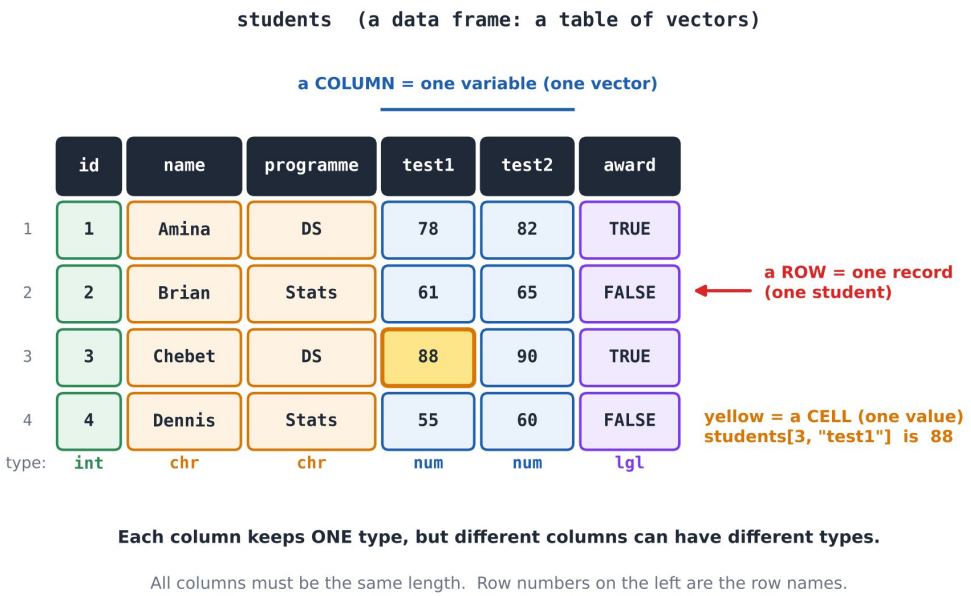


Figure 11: Anatomy of a data frame. Columns are vectors, rows are records, and each column has its own type.

> **In plain words:** You will use data frames more than any other structure. Almost every real dataset you ever load into R (from Excel, CSV files or a survey) arrives as a data frame.

6.2 Creating a data frame

Use data.frame() and give it one vector for each column, with the column name on the left of the equals sign. We will use this class register for most of this chapter, so type it carefully.

Code (type this)	What it does, in simple words	What you will see

`students <- data.frame(`
`  id = 1:6,`
`  name = c("Amina", "Brian", "Chebet", "Dennis", "Esther", "Farid"),`
`  programme = c("DS", "Stats", "DS", "Stats", "DS", "Stats"),`
`  test1 = c(78, 61, 88, 55, 92, 70),`
`  test2 = c(82, 65, 90, 60, 85, 74),`
`  award = c(TRUE, FALSE, TRUE, FALSE, TRUE, FALSE)`
	Builds a table with 6 columns and 6 rows. Each line is one column: a name, then an equals sign, then a vector. All the vectors must have the same length (6). Saved in the box students.	*(nothing is shown)*

Code (type this)	What it does, in simple words	What you will see
)		
students	Printing the data frame shows it as a table. The numbers on the left are row numbers.	

```
  id    name programme test1 test2 award
1  1   Amina        DS    78    82  TRUE
2  2   Brian     Stats    61    65 FALSE
3  3 Chebet        DS    88    90  TRUE
4  4 Dennis     Stats    55    60 FALSE
5  5 Esther        DS    92    85  TRUE
6  6   Farid     Stats    70    74 FALSE
```

> **Tip:** In older versions of R, text columns were turned into factors automatically (stringsAsFactors = TRUE). Since R 4.0.0 text stays as text, which is what we want. If you ever see odd factors, check your R version.

6.3 Checking your data: your first look

Whenever you meet a new dataset, spend a minute looking around before doing anything. These functions are your flashlight. Experienced analysts run them every single time.

Code (type this)	What it does, in simple words	What you will see
head(students, 3)	The first 3 rows. Perfect for a quick peek when a table has thousands of rows.	

```
  id    name programme test1 test2 award
1  1   Amina        DS    78    82  TRUE
2  2   Brian     Stats    61    65 FALSE
3  3 Chebet        DS    88    90  TRUE
```

tail(students, 2)	The last 2 rows.	

```
  id    name programme test1 test2 award
5  5 Esther        DS    92    85  TRUE
6  6   Farid     Stats    70    74 FALSE
```

dim(students)	Rows, then columns.	[1] 6 6
nrow(students)	Number of rows (students).	[1] 6
ncol(students)	Number of columns (variables).	[1] 6
names(students)	The column names.	

```
[1] "id"        "name"      "programme" "test1"     "test2"     "award"
```

class(students)	What kind of object is it?	[1] "data.frame"
is.data.frame(students)	A yes/no check.	[1] TRUE
str(students)	The most useful command of all. It shows the size, and for every column its type and the first few values. (int = whole number, chr = text, num = decimal number, logi = TRUE/FALSE.)	

```
'data.frame':  6 obs. of  6 variables:
 $ id       : int  1 2 3 4 5 6
 $ name     : chr  "Amina" "Brian" "Chebet" "Dennis" ...
 $ programme: chr  "DS" "Stats" "DS" "Stats" ...
 $ test1    : num  78 61 88 55 92 70
 $ test2    : num  82 65 90 60 85 74
 $ award    : logi  TRUE FALSE TRUE FALSE TRUE FALSE
```

Code (type this)	What it does, in simple words	What you will see
`summary(students)`	A summary of every column: for numbers the minimum, quartiles, median, mean and maximum; for text the length and type. Run this one yourself to see the full table.	
`(a summary table for every column)`		
`summary(students$test1)`	The summary for just one column (test1).	
`Min. 1st Qu. Median Mean 3rd Qu. Max.` `55.00 63.25 74.00 74.00 85.50 92.00`		
`class(students$test1)`	The type of one column.	`[1] "numeric"`
`is.numeric(students$test1)`	Is the column numbers?	`[1] TRUE`
`is.character(students$name)`	Is the column text?	`[1] TRUE`
`sapply(students[, 1:3], class)`	The type of each of the first three columns at once. (students[, 1:3] means "all rows, columns 1 to 3".)	
`id name programme` `"integer" "character" "character"`		
`anyNA(students)`	Is any value missing anywhere in the table?	`[1] FALSE`
`colSums(is.na(students))`	Count the missing values in each column. is.na() makes a table of TRUE/FALSE and colSums adds each column.	
`id name programme test1 test2 award` `0 0 0 0 0 0`		

> **Watch out:** Checking is not wasted time. Many analysis errors come from a column having the wrong type (numbers stored as text) or hidden missing values. str() and summary() catch these early.

6.4 Accessing: getting columns, rows and cells

A data frame has two directions, so brackets take two numbers: data[row, column]. You can also use the dollar sign for a column by name, which is the most popular way.

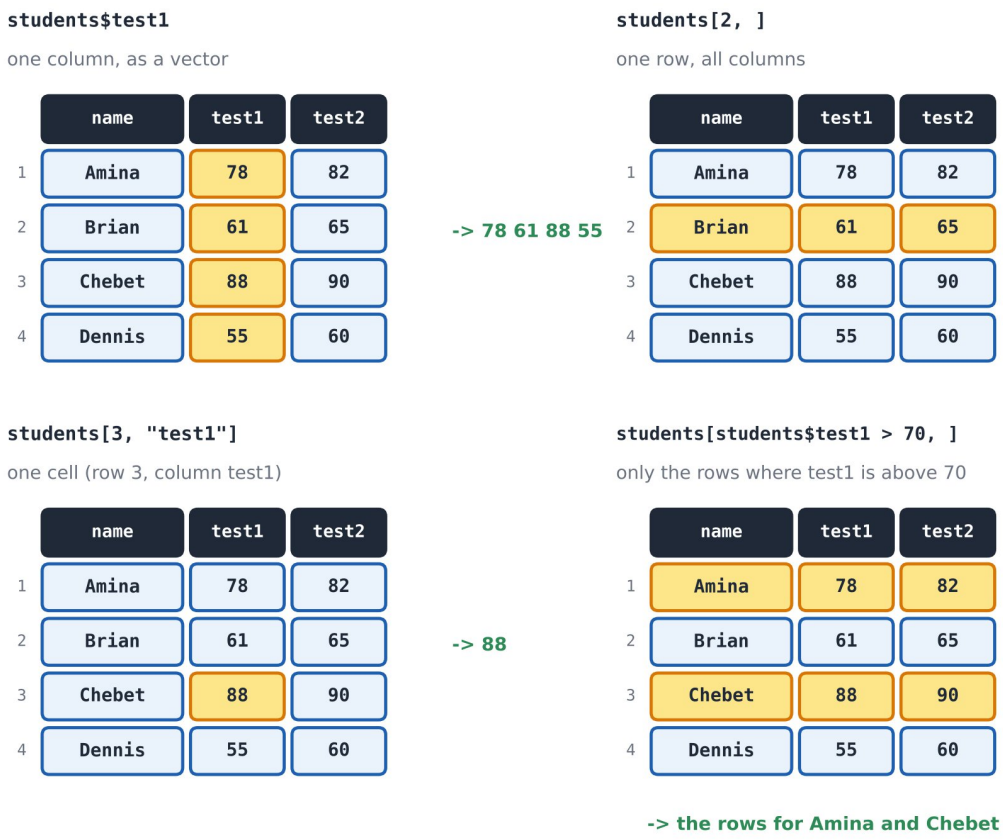


Figure 12: Four ways of picking from a data frame. Yellow shows what you get.

Code (type this)	What it does, in simple words	What you will see
`students$test1`	A whole column by name, using $. The result is a vector.	`[1] 78 61 88 55 92 70`
`students[["name"]]`	Double brackets with the name in quotes do the same job. Handy when the column name is stored in a variable.	
`[1] "Amina" "Brian" "Chebet" "Dennis" "Esther" "Farid"`		
`students[[2]]`	A column by position (the 2nd column, name).	
`[1] "Amina" "Brian" "Chebet" "Dennis" "Esther" "Farid"`		

`students["test1"]`	Single brackets with a name give back a smaller data frame (still a table), not a plain vector.	`test1`
		`1 78`
		`2 61`
		`3 88`
		`4 55`
		`5 92`
		`6 70`

`students[2, ]`	Row 2, all columns (nothing after the comma).	
`id name programme test1 test2 award` `2 2 Brian Stats 61 65 FALSE`		
`students[3, "test1"]`	One cell: row 3, column test1.	`[1] 88`

`students[1:2, c("name", "test1")]`	Several rows and several columns. c("name", "test1") picks two columns by name.	`name test1`
		`1 Amina 78`
		`2 Brian 61`

`students[, "test1"]`	All rows of one column. Because it is a single column, the answer comes back	`[1] 78 61 88 55 92 70`

Code (type this)	What it does, in simple words	What you will see
	as a vector.	

6.5 Filtering and sorting rows

Filtering means keeping only the rows that satisfy a condition. The recipe is: write a condition on a column (such as students$test1 > 75), put it before the comma, and R keeps the rows where the answer is TRUE.

Code (type this)	What it does, in simple words	What you will see
`students$test1 > 75`	The condition on its own: TRUE or FALSE for each student.	
`[1]  TRUE FALSE  TRUE FALSE  TRUE FALSE`		

`students[students$test1 > 75, c("name", "test1")]`	Keep rows where test1 is above 75, and show only the name and test1 columns.	`  name test1`
		`1  Amina    78`
		`3 Chebet    88`
		`5 Esther    92`

`students[students$programme == "DS" & students$test1 > 80, c("name", "test1")]`	Two conditions joined with & (and): in the DS programme AND test1 above 80. Note the double equals == for comparing.	`  name test1`
		`3 Chebet    88`
		`5 Esther    92`

`sum(students$test1 > 75)`	How many students scored above 75? (TRUE counts as 1.)	`[1] 3`

`subset(students, programme == "DS", select = c(name, test1, test2))`	subset() is a friendlier way to filter. The column names need no quotes or students$.	`  name test1 test2`
		`1  Amina    78    82`
		`3 Chebet    88    90`
		`5 Esther    92    85`

`students[order(-students$test1), c("name", "test1")]`	Sort from highest test1 to lowest. order() gives the row positions in sorted order; the minus sign reverses it.	`  name test1`
		`5 Esther    92`
		`3 Chebet    88`
		`1  Amina    78`
		`6  Farid    70`
		`2  Brian    61`
		`4 Dennis    55`

`head(students[order(students$test1), c("name", "test1")], 2)`	Sort from lowest up, then show just the first 2 rows: the two lowest scorers.	`  name test1`
		`4 Dennis    55`
		`2  Brian    61`

6.6 Adding rows

To add a row, create a one-row data frame with the same column names and glue it on with rbind(). The names must match exactly.

Code (type this)	What it does, in simple words	What you will see
`new_student <- data.frame(id = 7, name = "Grace", programme = "DS",` `  test1 = 80, test2 = 84, award = TRUE)`	A one-row data frame with the same six columns.	*(nothing is shown)*
`students_plus <- rbind(students, new_student)`	Stack the new row under the old table, and save as students_plus (so	*(nothing is shown)*

Code (type this)	What it does, in simple words	What you will see
	students stays at 6 rows for the next examples).	
`nrow(students_plus)`	Seven rows now.	`[1] 7`
`tail(students_plus, 1)`	Look at the last row.	
`  id  name programme test1 test2 award` `7  7 Grace        DS    80    84  TRUE`		
`products <- data.frame(id = c(1, 2), item = c("Pen", "Book"))`	A small table to show a classic trap.	*(nothing is shown)*
`bad <- rbind(products, c(3, "Ruler"))`	TRAP: c(3, "Ruler") mixes a number and text, so it becomes all text. When glued on, it drags the id column into text as well.	*(nothing is shown)*
`str(bad)`	Look: id is now chr (text), so you can no longer do maths with it.	

`'data.frame':  3 obs. of  2 variables:`
`$ id  : chr  "1" "2" "3"`
`$ item: chr  "Pen" "Book" "Ruler"`		

`good <- rbind(products, data.frame(id = 3, item = "Ruler"))`	FIX: add a one-row DATA FRAME, not a vector, so each column keeps its own type.	*(nothing is shown)*
`str(good)`	id stays numeric.	

`'data.frame':  3 obs. of  2 variables:`
`$ id  : num  1 2 3`
`$ item: chr  "Pen" "Book" "Ruler"`		

6.7 Adding columns, and arithmetic on columns

This is where data frames really shine. A column is a vector, so you can do vector maths on it, and you can store the answer as a brand-new column. R does the calculation for every row at once.

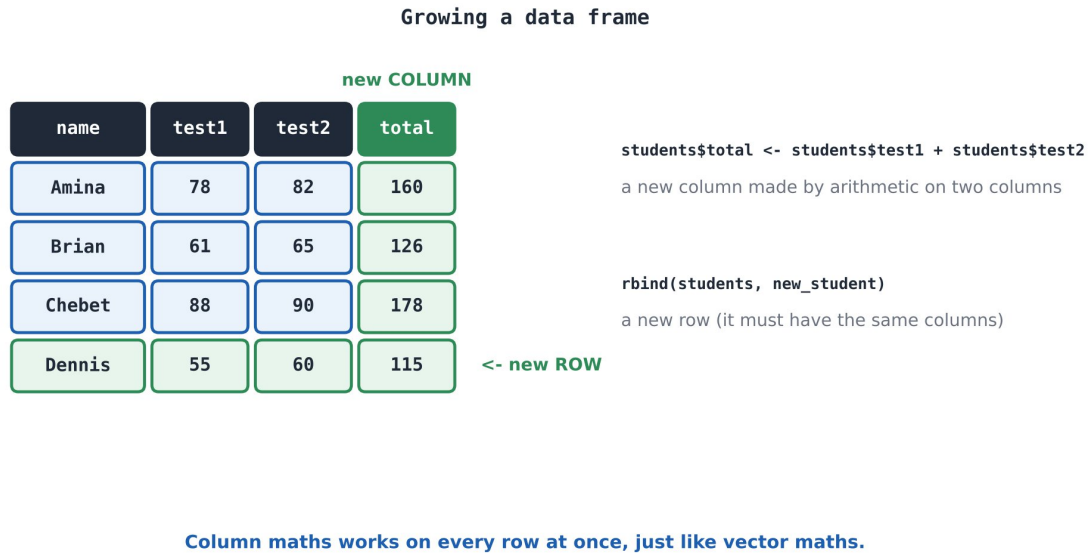


Figure 13: A new column is made by arithmetic on existing columns; a new row is glued on with rbind().

Code (type this)	What it does, in simple words	What you will see
`students$total <- students$test1 + students$test2`	Adds the two test columns row by row and stores the result in a NEW column called total. A new name after the $ creates a new column.	*(nothing is shown)*
`students$total`	Look at the new column: 78+82, 61+65, and so on.	
`[1] 160 126 178 115 177 144`		
`students$average <- students$total / 2`	Another new column: the average of the two tests.	*(nothing is shown)*
`students$average`	Check it.	
`[1] 80.0 63.0 89.0 57.5 88.5 72.0`		
`students$grade <- ifelse(students$average >= 70, "Pass", "Fail")`	ifelse() chooses for each row: "Pass" if the average is 70 or more, otherwise "Fail".	*(nothing is shown)*
`students$grade`	The result for each student.	
`[1] "Pass" "Fail" "Pass" "Fail" "Pass" "Pass"`		
`table(students$grade)`	Count how many Pass and how many Fail.	`Fail Pass` `2 4`
`round(students$test1 / 100, 2)`	Turn the marks into fractions of 100, rounded to 2 decimals.	
`[1] 0.78 0.61 0.88 0.55 0.92 0.70`		
`students$test2 - students$test1`	The improvement from test 1 to test 2 for each student. A negative number means a drop (Esther).	`[1] 4 4 2 5 -7 4`
`mean(students$test2 - students$test1)`	The average improvement across the class.	`[1] 2`
`sum(students$total)`	The sum of all the totals.	`[1] 900`
`students$name[which.max(students$total)]`	Who has the highest total? which.max() finds the row number, and that picks the name.	`[1] "Chebet"`
`students$award <- NULL`	Assigning NULL removes a column. We use this to drop the award column here.	*(nothing is shown)*
`names(students)`	award is gone and the new columns are there.	
`[1] "id" "name" "programme" "test1" "test2" "total"` `[7] "average" "grade"`		

> **Tip:** Column arithmetic follows the same rules as vector arithmetic: +, -, *, /, ^ and functions such as sqrt() and round() all work on a whole column at once.

6.8 Summaries and groups

Now we ask questions of the data: what is the average? how do the groups compare? These are the moments where a table turns into insight.

Code (type this)	What it does, in simple words	What you will see
`mean(students$test1)`	The average of one column.	`[1] 74`
`sapply(students[, c("test1", "test2")], mean)`	The average of several columns at once. sapply() applies mean to each chosen column.	`test1 test2` `74 76`
`colMeans(students[, c("test1", "test2")])`	A shortcut that gives the same answer for numeric columns.	`test1 test2` `74 76`
`sapply(students[, c("test1", "test2")], max)`	The highest mark in each test.	`test1 test2` `92 90`
`table(students$programme)`	How many students are in each programme?	`DS Stats` `3 3`

`aggregate(test1 ~ programme, data = students, FUN = mean)`	aggregate() groups the rows and calculates something for each group. Read the formula as "test1 by programme": the average test1 for each programme.	`programme test1`
		`1 DS 86`
		`2 Stats 62`

`tapply(students$test1, students$programme, mean)`	Another way to get the same group averages, as a named vector.	`DS Stats` `86 62`
`aggregate(cbind(test1, test2) ~ programme, data = students, FUN = mean)`	Group averages for two columns at once: cbind() lists the columns.	

`programme test1 test2`
`1 DS 86 85.66667`
`2 Stats 62 66.33333`		

`unique(students$programme)`	The different values in a column.	`[1] "DS" "Stats"`
`round(cor(students$test1, students$test2), 2)`	How strongly are the two tests related? A correlation near 1 means "students who do well in one also do well in the other".	`[1] 0.97`

Numbers become much clearer as pictures. The charts below show what R would draw for the same data. You can make them yourself with barplot() and plot(); we will study plotting in a later part.

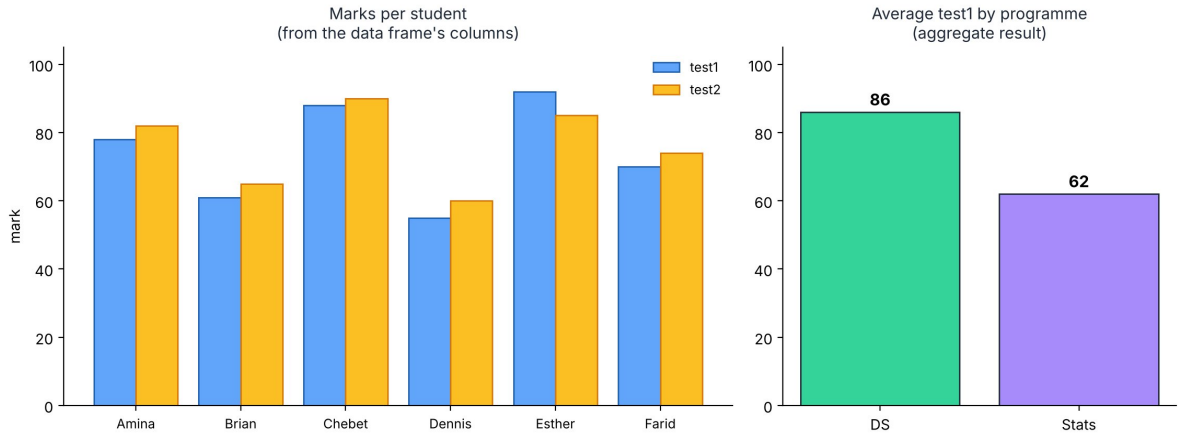


Figure 14: Left: marks of each student (from the columns). Right: group averages from aggregate().

Code (type this)	What it does, in simple words	What you will see
`barplot(students$test1, names.arg = students$name)`	Draws a bar chart of the test1 column, labelling each bar with the student's name. (A plot window opens in RStudio.)	`(a bar chart appears)`
`plot(students$test1, students$test2)`	A scatter plot: each dot is one student, with test1 across and test2 up. A cloud rising to the right shows a positive relationship.	
`(a scatter plot appears)`		

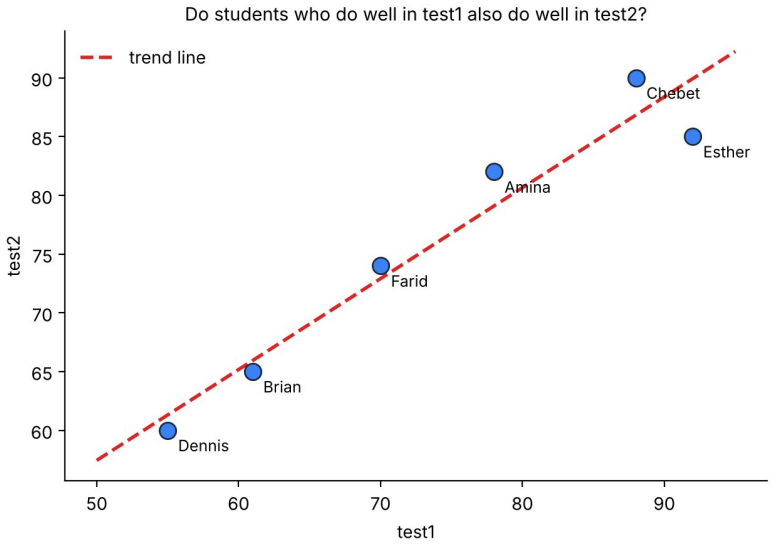


Figure 15: A scatter plot of the two tests. The dots rise together, matching a correlation of about 0.97.

6.9 Removing rows and columns

We use a small copy called tmp so that students stays complete.

Code (type this)	What it does, in simple words	What you will see
`tmp <- students[1:3, c("name", "test1", "test2")]`	Take the first 3 rows and 3 chosen columns into a small copy.	*(nothing is shown)*

`tmp`	Look at it.	`name test1 test2`
		`1 Amina 78 82`
		`2 Brian 61 65`
		`3 Chebet 88 90`

`tmp$test2 <- NULL`	Remove the test2 column.	*(nothing is shown)*

`tmp`	Only name and test1 are left.	`name test1`
		`1 Amina 78`
		`2 Brian 61`
		`3 Chebet 88`

`tmp[-2, ]`	Show the table without row 2. (A minus sign before the row number means "all except". Assign it back to keep the change.)	`name test1`
		`1 Amina 78`
		`3 Chebet 88`

`subset(tmp, test1 > 60)`	Or keep only the rows that meet a condition (here, scores above 60).	`name test1`
		`1 Amina 78`
		`3 Chebet 88`

`names(tmp)[2] <- "score"`	Rename the second column from	*(nothing is shown)*

Code (type this)	What it does, in simple words	What you will see
	test1 to score. names(tmp) is the vector of column names, and [2] picks the second.	
`names(tmp)`	Check the new names.	`[1] "name"  "score"`

6.10 Combining and joining data frames

There are three ways to put data frames together. Stack them (rbind) when they have the same columns. Place them side by side (cbind) when they have the same rows. Or join them by a shared key column (merge), like looking up each student's city from another table.

Code (type this)	What it does, in simple words	What you will see
`df1 <- data.frame(Name = c("Alice", "Bob"), Age = c(25, 30), Score = c(80, 75))`	The first table.	*(nothing is shown)*
`df2 <- data.frame(Name = c("Charlie", "David"), Age = c(28, 35), Score = c(90, 85))`	A second table with the same columns.	*(nothing is shown)*

`rbind(df1, df2)`	Stack vertically: the rows of df2 go under df1.	`Name Age Score`
		`1 Alice 25 80`
		`2 Bob 30 75`
		`3 Charlie 28 90`
		`4 David 35 85`

`df3 <- data.frame(Height = c(160, 175), Weight = c(55, 70))`	A table with different columns but the same number of rows as df1.	*(nothing is shown)*
`cbind(df1, df3)`	Place side by side: the columns of df3 are added to the right.	

`Name Age Score Height Weight`
`1 Alice 25 80 160 55`
`2 Bob 30 75 175 70`		

`cities <- data.frame(id = c(1, 2, 3), city = c("Nairobi", "Mombasa", "Kisumu"))`	A table of cities keyed by id.	*(nothing is shown)*

`merge(students[1:3, c("id", "name")], cities, by = "id")`	merge() joins two tables using the shared id column: each student gets their city.	`id name city`
		`1 1 Amina Nairobi`
		`2 2 Brian Mombasa`
		`3 3 Chebet Kisumu`

6.11 Missing values in a data frame

Code (type this)	What it does, in simple words	What you will see
`scores <- data.frame(name = c("A", "B", "C"), mark = c(70, NA, 85))`	A small table where B has no mark (NA).	*(nothing is shown)*
`is.na(scores$mark)`	Which marks are missing?	`[1] FALSE TRUE FALSE`
`colSums(is.na(scores))`	How many missing values in each column?	`name mark` `0 1`
`mean(scores$mark)`	One NA spoils the average.	`[1] NA`
`mean(scores$mark, na.rm = TRUE)`	Ignore the missing value.	`[1] 77.5`

Code (type this)	What it does, in simple words	What you will see

`na.omit(scores)`	Drop every row that has a missing value.	`  name mark`
		`1    A   70`
		`3    C   85`

`complete.cases(scores)`	TRUE for rows with no missing values.	`[1]  TRUE FALSE  TRUE`

`scores[scores$mark > 60, ]`	TRAP: filtering with a condition that hits an NA produces a strange row of NAs.	`   name mark`
		`1     A   70`
		`NA <NA>   NA`
		`3     C   85`

`subset(scores, mark > 60)`	FIX: subset() quietly leaves out rows where the condition is NA.	`  name mark`
		`1    A   70`
		`3    C   85`

6.12 Common data frame mistakes

Code (type this)	What it does, in simple words	What you will see
`data.frame(a = 1:3, b = 1:2)`	MISTAKE: the columns have different lengths (3 and 2). Every column must have the same number of rows. (Left out of the ready-made R file.)	
`Error in data.frame(a = 1:3, b = 1:2) :` `  arguments imply differing number of rows: 3, 2`		
`students$Test1`	MISTAKE: wrong capital letter. The real column is test1. R returns NULL instead of an error, so the mistake can hide.	`NULL`

`students[2]`	MISTAKE: meant row 2 but wrote only one number. Without a comma, R treats it as a COLUMN (the 2nd column, as a table). Write students[2,] for a row.	`    name`
		`1  Amina`
		`2  Brian`
		`3 Chebet`
		`4 Dennis`
		`5 Esther`
		`6  Farid`

`rbind(df1, data.frame(Nme = "Zed", Age = 40, Score = 60))`	MISTAKE: a typo in a column name (Nme instead of Name). rbind needs matching names. (Left out of the ready-made R file.)	
`Error in match.names(clabs, names(xi)) :` `  names do not match previous names`		

- **Using c() to add a row** turns numbers into text. Add a one-row data frame instead.
- **Forgetting the comma** in students[rows, columns].
- **Editing the only copy.** Make a backup, such as students_backup <- students, before big changes.
- **Filtering when the column has NA.** Use subset() or add !is.na(...) to your condition.
- **Column names with spaces.** They need backticks like students$`first name`; simple names are easier.

7. Tibbles

7.1 What is a tibble?

A tibble is a modern, tidier version of a data frame. It lives in the tibble package, which is part of the tidyverse (a popular family of R packages for working with data). A tibble is still a data frame underneath, so everything you learnt in Chapter 6 still works, but it behaves in a more careful, predictable way.

- **Nicer printing:** it shows only what fits on the screen (the first 10 rows), and shows the type of each column.
- **Fewer surprises:** it never changes your column names or turns text into factors, and it does not guess partial names.
- **Consistent subsetting:** picking one column with [ ] gives you a tibble again, not sometimes a vector.

7.2 Installing and loading

Installing a package is done once; loading it is done every time you start R. Think of installing as buying a book and loading as taking it off the shelf.

Code (type this)	What it does, in simple words	What you will see
`install.packages("tibble")`	Downloads and installs the tibble package. Do this once. (It is commented out in the ready-made R file; the file installs it automatically if missing.)	
`(a progress message appears)`		
`library(tibble)`	Loads the package so you can use its functions. Do this at the start of every session.	*(nothing is shown)*

7.3 Creating tibbles

There are three ways: tibble() from vectors, tribble() for typing a small table row by row, and as_tibble() to convert something you already have.

Way 1: tibble()

Code (type this)	What it does, in simple words	What you will see

`my_tib <- tibble(`
`  Id     = 1:4,`
`  Name   = c("Sandip", "Gaurav", "Ram", "Pratik"),`
`  Age    = c(25, 29, 30, 35),`
`  Role   = c("Engineer", "Data Scientist", "Developer", "HR"),`
`  Salary = c(45000, 60000, 80000, 100000)`
`)`
	Works just like data.frame(): one vector per column, all the same length. Saved as my_tib.	*(nothing is shown)*

`my_tib`	Look at the printing. The first line gives the size (4 rows by 5 columns). The small grey tags under the column names (<int>, <chr>, <dbl>) tell you each column's type.	

Code (type this)	What it does, in simple words	What you will see

`# A tibble: 4 × 5`
`Id Name Age Role Salary`
`<int> <chr> <dbl> <chr> <dbl>`
`1 1 Sandip 25 Engineer 45000`
`2 2 Gaurav 29 Data Scientist 60000`
`3 3 Ram 30 Developer 80000`
`4 4 Pratik 35 HR 100000`		

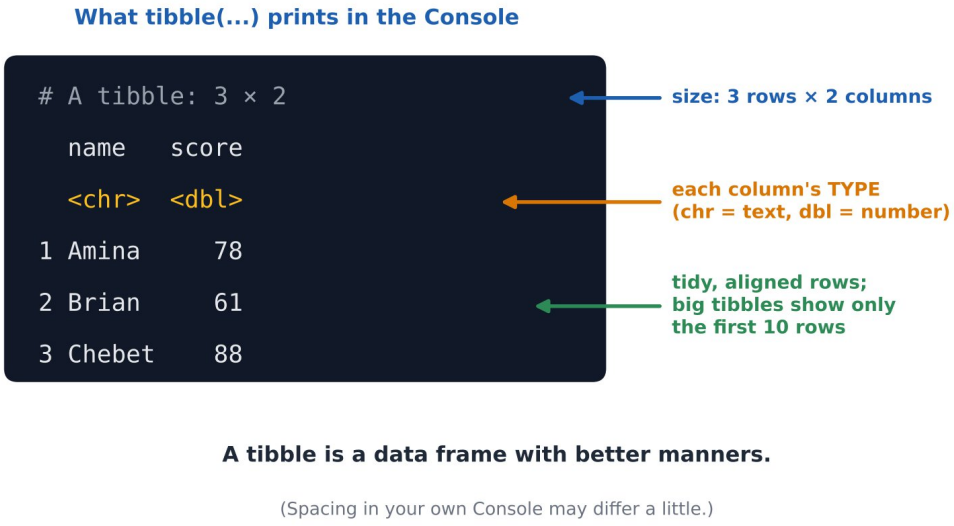


Figure 16: How a tibble prints. The type row (chr, dbl, int) is the big difference.

Way 2: tribble() (type a table row by row)

tribble() stands for "transposed tibble". You write the column names with a tilde (~) in front, then type the data row by row, just as it looks on paper. It is perfect for small tables.

Code (type this)	What it does, in simple words	What you will see

`people <- tribble(`
`~first_name, ~last_name, ~age, ~city,`
`"Saurabh", "Puri", 24, "pathardi",`
`"Prasad", "Bade", 22, "Beed",`
`"Manohar", "Khedkar", 27, "Ahmednagar"`
`)`
	The first line holds the column names (each starting with ~). Every line after is one row of data.	*(nothing is shown)*

`people`	A 3 by 4 tibble.	

`# A tibble: 3 × 4`
`first_name last_name age city`
`<chr> <chr> <dbl> <chr>`
`1 Saurabh Puri 24 pathardi`
`2 Prasad Bade 22 Beed`
`3 Manohar Khedkar 27 Ahmednagar`		

Way 3: as_tibble() (convert what you already have)

Code (type this)	What it does, in simple words	What you will see
`df <- data.frame(name = c("Saurabh", "Prasad", "Manohar"), age = c(25, 30, 35), city = c("New York", "San Francisco", "Los Angeles"))`	An ordinary data frame.	*(nothing is shown)*
`tib <- as_tibble(df)`	Convert the data frame into a tibble. The data stays exactly the same.	*(nothing is shown)*
`tib`	Same data, tidier print.	

`# A tibble: 3 × 3`
`  name      age city`
`  <chr>   <dbl> <chr>`
`1 Saurabh    25 New York`
`2 Prasad     30 San Francisco`
`3 Manohar    35 Los Angeles`		

`m <- matrix(1:6, nrow = 2, dimnames = list(NULL, c("a", "b", "c")))`	A matrix with column names (a tibble needs column names).	*(nothing is shown)*

`as_tibble(m)`	A matrix converts into a tibble too. Each matrix column becomes a tibble column.	`# A tibble: 2 × 3`
		`      a     b     c`
		`  <int> <int> <int>`
		`1     1     3     5`
		`2     2     4     6`

> **Tip:** If you convert a matrix with no column names, R may warn you or invent names V1, V2, V3. Give your matrix column names first, as above, to avoid this.

7.4 Checking a tibble

Code (type this)	What it does, in simple words	What you will see
`class(tib)`	A tibble has three class names, showing it is a data frame with extras.	
`[1] "tbl_df"     "tbl"        "data.frame"`		
`is.data.frame(tib)`	It is still a data frame, so all data frame tools work.	`[1] TRUE`
`dim(tib)`	Rows, then columns.	`[1] 3 3`
`names(tib)`	The column names.	
`[1] "name" "age"  "city"`		
`glimpse(tib)`	glimpse() is a tidyverse alternative to str(): it lists each column across the page with its type. (It comes with the tibble package.)	

`Rows: 3`
`Columns: 3`
`$ name <chr> "Saurabh", "Prasad", "Manohar"`
`$ age  <dbl> 25, 30, 35`
`$ city <chr> "New York", "San Francisco", "Los Angeles"`		

`as.data.frame(tib)`	Need a plain data frame again? Convert back. (We do not print it; it looks like an ordinary data frame.)	*(nothing is shown)*

7.5 Tibbles versus data frames: what really differs

Feature	Tibble	Data frame
Printing	Shows the first 10 rows and only the columns that fit; adds a type row	Prints everything, which can flood the screen
Column names	Kept exactly as you wrote them (even with spaces)	May be changed (spaces become dots)
Subsetting with [ ]	Always returns a tibble	Sometimes a vector, sometimes a data frame
Partial names	Does not guess: tib$na gives NULL with a warning	tab$na may silently match tab$name
Text columns	Never turned into factors	Modern R keeps text, but old versions turned them into factors
Where it lives	tibble package (tidyverse)	Base R (always available)

Code (type this)	What it does, in simple words	What you will see
`df_small <- data.frame(x = 1:3, y = c("a", "b", "c"))`	A tiny data frame.	*(nothing is shown)*
`df_small[, "x"]`	Data frame: picking one column with [,] gives a plain vector.	`[1] 1 2 3`
`tib_small <- as_tibble(df_small)`	The same data as a tibble.	*(nothing is shown)*

`tib_small[, "x"]`	Tibble: picking one column still gives a tibble. This consistency prevents nasty surprises.	`# A tibble: 3 × 1`
		`  x`
		`  <int>`
		`1  1`
		`2  2`
		`3  3`

`tib_small$x`	If you really want the vector from a tibble, use $ or [[ ]].	`[1] 1 2 3`
`df_n <- data.frame(name = c("A", "B"))`	A data frame with a column called name.	*(nothing is shown)*
`df_n$na`	Data frame: R guesses and matches the partial name "na" to "name". Convenient but risky.	`[1] "A" "B"`
`as_tibble(df_n)$na`	Tibble: no guessing. You get NULL and a warning (Unknown or uninitialised column).	`NULL`
`names(data.frame("my col" = 1))`	Data frame: the space in the name was changed into a dot.	`[1] "my.col"`
`names(tibble("my col" = 1))`	Tibble: the name is kept exactly as you typed it.	`[1] "my col"`

7.6 A taste of the tidyverse (optional)

Tibbles are designed to work smoothly with the dplyr package, which gives friendly verbs for data jobs: filter() to keep rows, mutate() to add columns, select() to pick columns. The symbol %>% (the

pipe) means "then" and passes the result on to the next step. You can ignore this section for now and come back later.

Code (type this)	What it does, in simple words	What you will see
`library(dplyr)`	Loads dplyr (install it once with install.packages("dplyr")).	*(nothing is shown)*
`scores_tib <- tibble(name = c("Amina", "Brian", "Chebet"), score = c(78, 61, 88))`	A small tibble.	*(nothing is shown)*

`scores_tib %>% filter(score > 70)`	Take scores_tib, THEN keep only the rows where score is above 70.	`# A tibble: 2 × 2`
		`  name   score`
		`  <chr>  <dbl>`
		`1 Amina     78`
		`2 Chebet    88`

`scores_tib %>% mutate(grade = if_else(score >= 70, "Pass", "Fail"))`	Take scores_tib, THEN add a new column called grade.	`# A tibble: 3 × 3`
		`  name   score grade`
		`  <chr>  <dbl> <chr>`
		`1 Amina     78 Pass`
		`2 Brian     61 Fail`
		`3 Chebet    88 Pass`

8. How the Data Structures Differ

You have now met seven containers. Here is the big comparison. Keep this page nearby while you practise.

8.1 The comparison table

Structure	Shape	Types inside	Create with	Pick items with
Vector	1D row	ONE type	c(1, 2, 3)	x[2]
Factor	1D row	Categories (levels)	factor(c("a", "b"))	f[2]
Matrix	2D grid	ONE type	matrix(1:6, nrow = 2)	m[row, col]
Array	3D or more	ONE type	array(1:12, dim = c(2, 3, 2))	a[row, col, layer]
List	1D bundle	MIXED, any shape	list(name = "Ana", age = 21)	l$name l[[1]] l["name"]
Data frame	2D table	Each COLUMN has its own type	data.frame(x = 1:3, y = c("a", "b", "c"))	df$x df[row, col]
Tibble	2D table	Each COLUMN has its own type	tibble(x = 1:3, y = c("a", "b", "c"))	tb$x tb[row, col] (stays a tibble)

8.2 Key differences in plain words

- **Vector versus list:** a vector must hold one type; a list can hold anything in each compartment.
- **Vector versus matrix:** a vector is one row of values; a matrix is a grid, with the same one-type rule.
- **Matrix versus array:** a matrix has 2 dimensions (rows, columns); an array can have 3 or more (adding layers).
- **Matrix versus data frame:** a matrix holds one type everywhere (all numbers or all text); a data frame lets each column have its own type, which is how real datasets look.
- **Data frame versus list:** a data frame is a list whose items (columns) must all be the same length.
- **Data frame versus tibble:** same idea, but the tibble prints more tidily and behaves more predictably.
- **Factor versus character vector:** a factor remembers the fixed set of allowed groups (levels) and stores them as codes; a character vector is just free text.

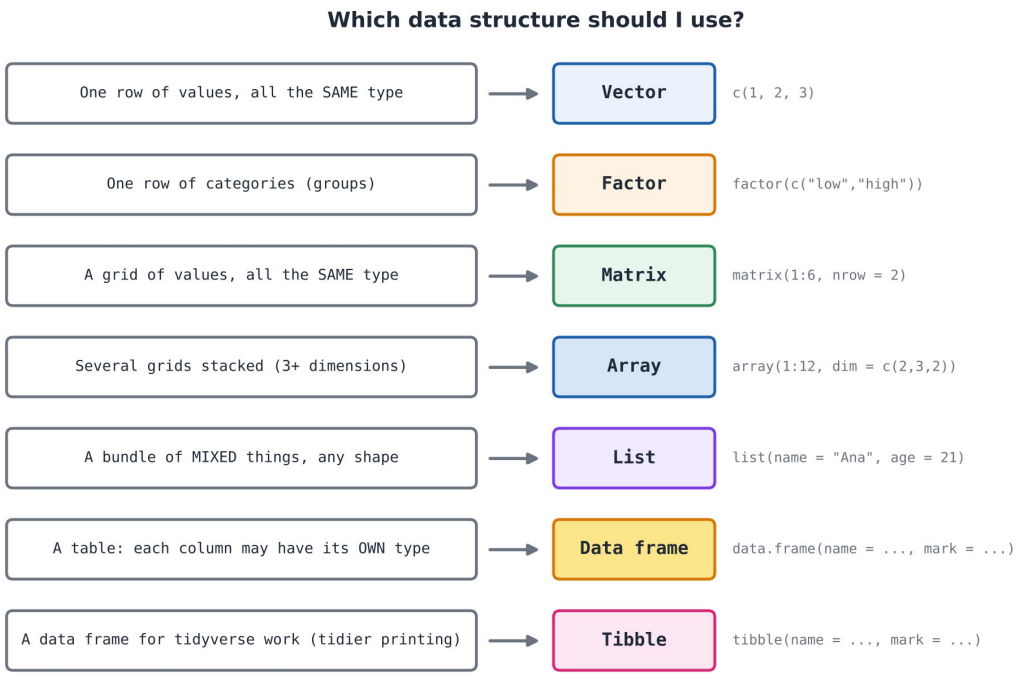


Figure 17: A quick guide to choosing the right structure.

8.3 Seeing the differences in code

Code (type this)	What it does, in simple words	What you will see
`v <- c(1, 2, 3)`	A vector.	*(nothing is shown)*
`f <- factor(c("a", "b"))`	A factor.	*(nothing is shown)*
`m <- matrix(1:4, nrow = 2)`	A matrix.	*(nothing is shown)*
`a <- array(1:8, dim = c(2, 2, 2))`	An array.	*(nothing is shown)*
`l <- list(1, "a")`	A list.	*(nothing is shown)*
`d <- data.frame(x = 1:2)`	A data frame.	*(nothing is shown)*
`tb <- tibble(x = 1:2)`	A tibble.	*(nothing is shown)*
`class(v)`	The class of the vector.	`[1] "numeric"`
`class(f)`	The class of the factor.	`[1] "factor"`
`class(m)`	A matrix is also an array.	`[1] "matrix" "array"`
`class(a)`	The array.	`[1] "array"`
`class(l)`	The list.	`[1] "list"`
`class(d)`	The data frame.	`[1] "data.frame"`
`class(tb)`	The tibble carries three names.	
`[1] "tbl_df"  "tbl"  "data.frame"`		
`dim(v)`	A vector has no dimensions (NULL). Use length() instead.	`NULL`
`dim(m)`	The matrix: 2 rows, 2 columns.	`[1] 2 2`

Code (type this)	What it does, in simple words	What you will see
`dim(a)`	The array: 2 rows, 2 columns, 2 layers.	`[1] 2 2 2`
`dim(d)`	The data frame: 2 rows, 1 column.	`[1] 2 1`
`length(l)`	The list: 2 compartments.	`[1] 2`
`c(1, "a")`	A vector with mixed types: R forces everything into text.	`[1] "1" "a"`

`str(list(1, "a"))`	A list with mixed types: each item keeps its own type.	`List of 2`
		` $ : num 1`
		` $ : chr "a"`

`str(data.frame(n = 1:2, s = c("a", "b")))`	A data frame: each COLUMN keeps its own type.	

```
'data.frame':  2 obs. of  2 variables:
 $ n: int  1 2
 $ s: chr  "a" "b"
```

9. Practice Exercises

Learning to code is like learning to swim: you have to get in the water. Try each exercise yourself before checking the answers. The practice file that comes with this document has the starting data for every exercise, with space for your code.

Part A: Think about it (no code)

- A1. Which structure would you use to store the marks of five students in one subject?
- A2. Which structure would you use to keep one patient's name, age and a list of temperature readings together?
- A3. Which structure suits survey answers like Yes / No / Maybe?
- A4. In one sentence, what is the difference between a matrix and a data frame?
- A5. What is the difference between student_info["age"] and student_info[["age"]]?
- A6. In a matrix called m, what do m[2,] and m[, 2] give you?
- A7. Name two ways a tibble behaves differently from a data frame.

Part B: Coding exercises

- B1. Make a 3 x 4 matrix called m from the numbers 1 to 12, filled row by row. Find its dimensions, then pick out row 2 and column 4.
- B2. Multiply every value in m by 10 and find the sum of each row of the result.
- B3. Add a new row (13, 14, 15, 16) to m. What are the dimensions now?
- B4. Create arr <- array(1:24, dim = c(2, 3, 4)). What is the value in row 2, column 3, layer 4?
- B5. Create a list pet with name "Mimi", age 3, vaccinated TRUE and toys c("ball", "rope"). Show the 2nd toy, add weight = 4.5, remove age, then show the names.
- B6. ratings <- c("good", "poor", "good", "excellent", "poor", "good"). Make an ordered factor (poor < good < excellent) and count each level.
- B7. Make a data frame of 4 products: item (Pen, Book, Bag, Cap), price (50, 120, 800, 250), stock (100, 40, 15, 60). Add a column value = price x stock. Show the products whose value is above 5000.
- B8. x <- c(12, NA, 15, 9, NA). Count the missing values, find the mean ignoring them, then replace the NAs with that mean.
- B9. Join these two tables by id: a = (1 Ann, 2 Bob, 3 Cy) and b = (id 1 Nairobi, id 3 Kisumu).
- B10. (Needs the tibble package.) Turn the products data frame from B7 into a tibble and print it.

Part C: Real-life application questions

- C1. School: using the students data frame, which programme has the higher average on test2, DS or Stats?
- C2. Shop inventory: store the stock of 3 products (Pen, Book, Bag) in 2 shops (ShopA: 10, 5, 8; ShopB: 7, 12, 3) in a matrix with names. Find the total stock of each product and of each shop.
- C3. Survey: answers <- c("Yes", "No", "Yes", "Maybe", "Yes", "No"). Turn them into a factor, count each answer, and give the percentage for each (rounded).

- C4. Hospital: patient is a list with id "P01", name "Zawadi" and temps c(36.8, 37.5, 38.2). Find the highest temperature, check whether any reading is above 38, and add a status item that says "Fever" if the highest is above 38, else "Normal".
- C5. Business: a data frame sales has month (Jan, Feb, Mar), revenue (1200, 1500, 1100) and cost (800, 900, 950). Add a profit column, find the best month, and the total profit.

Answers: Part A

- A1. A vector, for example c(55, 70, 82, 64, 91).
- A2. A list, because it holds mixed types (text, number, vector) in separate compartments.
- A3. A factor, because the answers come from a small fixed set of groups.
- A4. A matrix holds one type of value everywhere, while a data frame lets each column have its own type.
- A5. Single brackets return a smaller list that still has the label; double brackets return the value itself.
- A6. m[2,] is row 2 (all columns); m[, 2] is column 2 (all rows).
- A7. Any two of: tidier printing (first 10 rows and a type row), never changes column names, never guesses partial names, [ ] always returns a tibble.

Answers: Part B (code and output)

Code (type this)	What it does, in simple words	What you will see

`# B1`
`m <- matrix(1:12, nrow = 3, byrow = TRUE)`
`dim(m)`
`m[2, ]`
`m[, 4]`
	Fill by row, then check the size, row 2 and column 4.	`[1] 3 4`
		`[1] 5 6 7 8`
		`[1]  4  8 12`

`# B2` `rowSums(m * 10)`	Multiply every cell by 10 first, then add up each row.	`[1] 100 260 420`
`# B3` `dim(rbind(m, 13:16))`	13:16 is the new row (13, 14, 15, 16). Now 4 rows and 4 columns.	`[1] 4 4`

`# B4`
`arr <- array(1:24, dim = c(2, 3, 4))`
`arr[2, 3, 4]`
	The last cell of the last layer holds the last number.	`[1] 24`

`# B5`
`pet <- list(name = "Mimi", age = 3, vaccinated = TRUE, toys = c("ball", "rope"))`
`pet$toys[2]`
`pet$weight <- 4.5`
`pet$age <- NULL`
`names(pet)`
	Get the 2nd toy, add weight, delete age, then list the names.	

`[1] "rope"` `[1] "name"  "vaccinated"  "toys"  "weight"`		

`# B6`
`ratings <- c("good", "poor", "good", "excellent", "poor", "good")`
`ratings_f <- factor(ratings, levels = c("poor", "good", "excellent"), ordered = TRUE)`
`table(ratings_f)`
	Put the levels in the right order, mark the factor as ordered, then count.	

`ratings_f`
`poor  good excellent`
`2  3  1`		

`# B7` `shop <- data.frame(item = c("Pen", "Book", "Bag", "Cap"),`	Make the table, add the value column (price times stock), then filter. Pen (5000) and Book (4800) are not above 5000.	

Code (type this)	What it does, in simple words	What you will see

`price = c(50, 120, 800, 250), stock = c(100, 40, 15, 60))`
`shop$value <- shop$price * shop$stock`
`shop[shop$value > 5000, ]`		

`item price stock value`
`3 Bag 800 15 12000`
`4 Cap 250 60 15000`		

`# B8`
`x <- c(12, NA, 15, 9, NA)`
`sum(is.na(x))`
`mean(x, na.rm = TRUE)`
`x[is.na(x)] <- mean(x, na.rm = TRUE)`
`x`
	Count the NAs, find the mean of the rest (12), then fill the holes with it.	`[1] 2`
		`[1] 12`
		`[1] 12 12 15 9 12`

`# B9`
`a <- data.frame(id = 1:3, name = c("Ann", "Bob", "Cy"))`
`b <- data.frame(id = c(1, 3), city = c("Nairobi", "Kisumu"))`
`merge(a, b, by = "id")`
	Only ids that appear in both tables (1 and 3) are kept.	`id name city`
		`1 1 Ann Nairobi`
		`2 3 Cy Kisumu`

`# B10` `as_tibble(shop)`	Convert the products data frame (from B7) into a tibble.	

`# A tibble: 4 × 4`
`item price stock value`
`<chr> <dbl> <dbl> <dbl>`
`1 Pen 50 100 5000`
`2 Book 120 40 4800`
`3 Bag 800 15 12000`
`4 Cap 250 60 15000`		

Answers: Part C (code and output)

Code (type this)	What it does, in simple words	What you will see
`# C1` `tapply(students$test2, students$programme, mean)`	Average test2 for each programme. DS is higher.	`DS Stats` `85.66667 66.33333`

`# C2`
`stock <- matrix(c(10, 5, 8, 7, 12, 3), nrow = 3,`
`dimnames = list(c("Pen", "Book", "Bag"), c("ShopA", "ShopB")))`
`stock`
`rowSums(stock)`
`colSums(stock)`
	The values fill down the columns: ShopA gets 10, 5, 8 and ShopB gets 7, 12, 3. rowSums totals each product; colSums totals each shop.	`ShopA ShopB`
		`Pen 10 7`
		`Book 5 12`
		`Bag 8 3`
		`Pen Book Bag`
		`17 17 11`
		`ShopA ShopB`
		`23 22`

`# C3`
`answers <- c("Yes", "No", "Yes", "Maybe", "Yes", "No")`
`answers_f <- factor(answers)`
`table(answers_f)`
`round(prop.table(table(answers_f)) * 100)`
	Count each answer, then turn the counts into percentages (prop.table gives fractions of the total).	`answers_f`
		`Maybe No Yes`
		`1 2 3`
		`answers_f`
		`Maybe No Yes`
		`17 33 50`

`# C4`
`patient <- list(id = "P01", name = "Zawadi", temps = c(36.8, 37.5, 38.2))`
	The highest reading, a yes/no check, then a new list item decided by ifelse().	`[1] 38.2`
		`[1] TRUE`
		`[1] "Fever"`

Code (type this)	What it does, in simple words	What you will see

`max(patient$temps)`
`any(patient$temps > 38)`
`patient$status <-`
`ifelse(max(patient$temps) > 38,`
`"Fever", "Normal")`
`patient$status`		

`# C5`
`sales <- data.frame(month = c("Jan",`
`"Feb", "Mar"),`
`  revenue = c(1200, 1500, 1100),`
`cost = c(800, 900, 950))`
`sales$profit <- sales$revenue -`
`sales$cost`
`sales$profit`
`sales$month[which.max(sales$profit)]`
`sum(sales$profit)`
	Profit per month, the month with the biggest profit, and the total.	`[1] 400 600 150`
		`[1] "Feb"`
		`[1] 1150`

10. Free Resources to Keep Learning

All of these are free to read online. The first group is the best place to start.

Free books

- **R for Data Science (2nd edition):** https://r4ds.hadley.nz Friendly, practical, written for beginners. Great chapters on data import, tidying, tibbles and transformation.
- **Hands-On Programming with R:** https://rstudio-education.github.io/hopr Gentle introduction to R objects, including vectors, matrices, arrays, lists and data frames.
- **ModernDive (Statistical Inference via Data Science, 2nd edition):** https://moderndive.com/v2/ Learn R and data frames step by step while doing statistics.
- **R Programming for Data Science (Roger Peng):** https://bookdown.org/rdpeng/rprogdatascience/ Clear explanations of R's data types and structures.
- **Advanced R, Vectors chapter (Hadley Wickham):** https://adv-r.hadley.nz/vectors-chap.html For later: a deeper look at vectors, lists, data frames and tibbles.
- **An Introduction to R (official manual):** https://cran.r-project.org/doc/manuals/r-release/R-intro.html The official reference; good for looking things up.

Tibbles and the tidyverse

- **tibble package website:** https://tibble.tidyverse.org/ Official documentation and examples for tibbles.
- **dplyr package website:** https://dplyr.tidyverse.org/ Official guide to filter(), mutate(), select() and more.
- **R4DS: Data transformation:** https://r4ds.hadley.nz/data-transform Learn to work with tibbles using dplyr.
- **R4DS: Workflow basics:** https://r4ds.hadley.nz/workflow-basics Good habits for writing R code.

Practice and community

- **swirl: learn R inside R:** https://swirlstats.com/ Interactive lessons in your Console.
- **Posit Cloud (R in your browser):** https://posit.cloud/ No installation needed; free for personal use.
- **Posit Education: Beginners:** https://education.rstudio.com/learn/beginner/ A curated learning path for new R learners.
- **Posit Community forum:** https://forum.posit.co/ Friendly place to ask questions.
- **Install R:** https://cloud.r-project.org/ Download R (install first).
- **Install RStudio Desktop:** https://posit.co/download/rstudio-desktop/ Download the RStudio workspace (install second).

> **About these links:** The examples in these notes were adapted from common R tutorials such as those on GeeksforGeeks (geeksforgeeks.org), with corrections and extra explanations. The book and tibble links were checked when this document was prepared; if a link does not open, search for the title and author's name.

Where to go next

You now know the containers R uses to hold data. Next steps: loading real data from CSV files (read.csv), drawing charts with ggplot2, writing your own functions and loops, and exploring the tidyverse with R for Data Science. Keep practising a little every day. You are doing wonderfully.